INFORME DE INVESTIGACIÓN · OCTUBRE 2026

Seguridad del desarrollo de software con inteligencia artificial

Estado del arte, bases open source y decisión de producto

Una propuesta local, determinista y verificable para revisar cambios y dependencias, partiendo de herramientas existentes.

Fecha de corte: 28 de octubre de 2026
Alcance: investigación técnica, producto, costos, roadmap y validación

Índice

1. Resumen ejecutivo y decisión 4

1.1 Qué construir primero 4

1.2 Por qué podría importar 4

1.3 Lo que ya está resuelto o parcialmente resuelto 5

1.4 Recomendación económica y de alcance 5

2. Método y estado de la evidencia 6

2.1 Preguntas y selección de fuentes 6

2.2 Límites de esta investigación 6

3. Estado del arte: asistentes, modelos y agentes 7

3.1 Superficies comunes 7

3.2 Modelos propietarios, pesos abiertos y modelos pequeños 9

3.3 Incidentes observados frente a riesgos generales 9

4. Mapa de amenazas y controles 10

4.1 Código generado o modificado 10

4.2 Agentes, MCP y confianza 11

4.3 Dependencias y cadena de suministro 11

4.4 Infraestructura y estándares 12

5. Análisis competitivo 13

5.1 Herramientas de seguridad y madurez 13

5.2 Competidor más cercano: lectura crítica de OpenQodex 16

6. Catálogo y evaluación de bases reutilizables 17

6.1 Leyenda y criterios 17

6.2 Otras piezas y estrategias 21

6.3 Tres arquitecturas posibles sobre bases existentes 22

7. Oportunidades y priorización 25

7.1 Criterios 25

8. Tres propuestas y decisión 29

9. Arquitectura recomendada 30

9.1 Principios y límites verificables 30

9.2 Componentes, entradas y salidas 31

9.3 Snapshot, alcance y caché 32

9.4 Flujo de análisis 33

9.5 Remediación y rollback: fase posterior . . . 34
9.6 Agentes especializados: decisión condicionada . . . 35
9.7 Seguridad de la herramienta . . . 35

10. Integraciones, notificaciones y automatización . . . 37

10.1 Prioridad y límites . . . 37
10.2 Notificaciones . . . 38
10.3 Qué puede automatizarse . . . 38

11. Costos, infraestructura y tokens . . . 39

11.1 Precios verificables y fórmula . . . 39
11.2 Comparación técnica: escenarios hipotéticos . . . 40
11.3 Presupuesto operativo mensual ilustrativo . . . 41
11.4 Presupuesto de desarrollo y mantenimiento . . . 42
11.5 Reducir consumo sin degradar silenciosamente cobertura . . . 42

12. MVP, requisitos y aceptación . . . 43

12.1 Alcance . . . 43
12.2 Historias y criterios verificables . . . 43

13. Plan de validación y pruebas de concepto . . . 44

13.1 Evidencia disponible y conflictos . . . 44
13.2 Prueba de concepto ejecutada aquí . . . 44
13.3 Experimentos reproducibles pendientes . . . 45
13.4 Métricas y diseño científico . . . 45

14. Licencias, comunidad y sostenibilidad . . . 47

14.1 Comparación . . . 47
14.2 Gobierno, seguridad y comunidad . . . 47

15. Roadmap y backlog . . . 48

15.1 Plan de fases . . . 48
15.2 Primeras cuatro semanas . . . 49
15.3 Backlog inicial listo para trasladar . . . 51
15.4 Experimentos de demanda y razones para fallar . . . 52

16. Decisión final y próximas acciones . . . 53

16.1 Respuestas obligatorias . . . 53
16.2 Tecnologías seleccionadas para el spike . . . 54

17. Bibliografía y fuentes . . . 56

18. Apéndice: reproducir la prueba local ejecutada . . . 59

1. Resumen ejecutivo y decisión

1.1 Qué construir primero

Construir una **prueba de integración local y determinista** que amplíe el modo de escaneo de OpenQodex con OSV-Scanner y una base de avisos descargada previamente. Debe indicar explícitamente qué snapshot de código y lockfiles se revisó, con qué escáneres y datos, si faltó cobertura y cuán antiguos son los avisos. La salida será útil para una persona, un hook y un asistente mediante JSON/SARIF. No necesita LLM, servicio remoto ni credenciales para analizar una vez preparados sus insumos.

La aportación no es «inventar un orquestador». OpenQodex ya selecciona escáneres según los archivos modificados, revisa lockfiles, genera SARIF y tiene un hook que verifica los commits concretos de un push ejecutado con hooks. Su documentación dice que su `--offline` omite OSV-Scanner y otros escáneres que usan red, y que no tiene una copia offline de los avisos. OSV-Scanner, por separado, sí soporta una base local y `--offline`. La hipótesis es conectar ambas capacidades y hacer explícita la cobertura, no atribuir a nuestra idea funciones que el competidor ya tiene [S08, S14, S51].

Condición de decisión: antes de una plataforma nueva, reproducir OpenQodex tal cual y evaluar si una contribución pequeña puede satisfacer los casos. Si upstream ya resuelve el problema, contribuir, documentar o usarlo. Si la integración es inviable o rechazada, considerar un adaptador independiente compatible con sus formatos. No se ha auditado su código ni probado su integración en este entorno; las afirmaciones de funciones proceden de documentación del proyecto, no de una verificación independiente.

1.2 Por qué podría importar

- El 84 % de quienes respondieron la pregunta de la encuesta de Stack Overflow 2025 dijo usar o planear usar IA en desarrollo. Es un indicador de adopción, no una medición de demanda para este producto [S07].
- Hay investigación que encontró problemas de seguridad en código generado o en tareas asistidas, pero sus muestras, modelos y ejercicios son anteriores y no describen automáticamente a los modelos actuales. Otros estudios encuentran resultados distintos según lenguaje y tarea [S04–S06].
- Existen quejas y reportes de falsos positivos en escáneres reales; son evidencia cualitativa de fricción, no una estimación poblacional ni una promesa de que nuestra capa la reduzca [S56–S57].
- El modo local sin red, los datos de vulnerabilidades fechados y una salida que declare análisis incompleto pueden ser útiles para equipos con repositorios privados, conexión intermitente o restricciones de datos. **Que estos usuarios prefieran y adopten esta solución es una hipótesis aún no validada.**

1.3 Lo que ya está resuelto o parcialmente resuelto

Necesidad	Estado observado	Implicación
Ejecutar varios escáneres con un comando	Ya existen SecGate y OpenQodex [S08–S09]	No es diferenciación suficiente.
Escanear dependencias sin conexión	OSV-Scanner ofrece modo offline con base local [S51]	Integrar y gestionar su vigencia; no crear otra base.
Revisar el cambio de agentes y hooks	OpenQodex ya lo hace en escenarios documentados [S08]	Medir sus límites y reutilizar sus mecanismos.
Controlar servidores o tráfico MCP	ToolHive, Pipelock y otros ya lo intentan [S44–S45]	No añadir un firewall universal en el MVP.
Saber que un resultado es seguro y completo	Ninguna herramienta evaluada demuestra cobertura universal	Declarar alcance e incertidumbre; no prometer ausencia de vulnerabilidades.

1.4 Recomendación económica y de alcance

Estimación de trabajo, no cotización: **4 semanas de calendario para un prototipo evaluable con dos personas a 20 h/semana cada una; 8–12 semanas para un MVP usable**, según resultados de integración y seguridad. A una tarifa interna de referencia explícita de USD 25–50 por hora, 160 h de prototipo son USD 4,000–8,000; 320–480 h de MVP son USD 8,000–24,000. Son supuestos de planeación, no salarios investigados ni costos inevitables; trabajo de fundadores tiene costo de oportunidad. Infraestructura local y llamadas de IA pueden ser **USD 0 en cargos de proveedor**, pero hardware, energía, conexión inicial, mantenimiento y tiempo humano no son gratis. Ver secciones 11 y 15.

La prueba controlada de esta investigación ejecutó detectores didácticos sobre un repositorio sintético: el análisis total encontró cuatro candidatos y el incremental tres en dos archivos modificados. No se ejecutó Semgrep, OSV-Scanner ni OpenQodex, porque no estaban instalados y el entorno de ejecución no resolvió DNS hacia repositorios o registros públicos. Se consultó un aviso OSV mediante fuentes públicas; se usó una copia mínima manual para el experimento. No extrapolar tiempos ni precisión de esta prueba al producto [S50].

2. Método y estado de la evidencia

2.1 Preguntas y selección de fuentes

Se contrastaron documentación oficial, repositorios originales y sus archivos de licencia, avisos de seguridad, estudios académicos, benchmarks y comentarios públicos. Se priorizó el texto de los mantenedores y las especificaciones. Las fichas comerciales describen capacidades declaradas, no pruebas independientes. Las búsquedas y documentos se revisaron con fecha de corte 28/10/2026; versiones, precios y estado pueden cambiar. Los datos de mantenimiento se apoyan en señales visibles de commits, releases, documentación y avisos en esa fecha, **sin auditoría cuantitativa de cada historial**. En varios repositorios solo pudo verificarse una señal de actividad reciente y no su frecuencia exacta: se indica como tal.

Etiquetas: hecho verificado en una fuente; resultado de investigación o demostración con su alcance; estimación de ingeniería; hipótesis de producto; y testimonio cualitativo. Los resultados del proveedor no se presentan como comparaciones independientes. Donde no hay datos no se inventan benchmarks, cobertura, tasa de falsos positivos, precios o estados de mantenimiento.

2.2 Límites de esta investigación

El entorno permite consultar páginas públicas, ejecutar Git y Python y generar documentos, pero no descargar herramientas desde GitHub, npm, PyPI o la API OSV a través del proceso local. Por ello no hay prueba directa de bases candidatas, vulnerabilidades reales en aplicaciones, inferencia LLM ni rendimiento real. Un sitio público puede describir funciones que se comporten distinto según versión, plataforma, plan o configuración. La evaluación legal es informativa y requiere revisión de las licencias y dependencias de la versión distribuida. No se han entrevistado usuarios ni estudiado su disposición a pagar.

3. Estado del arte: asistentes, modelos y agentes

3.1 Superficies comunes

Un asistente puede recibir texto de chat, repositorio, instrucciones, archivos, resultados de herramientas, documentación web o MCP; luego proponer y, según permisos, escribir archivos, ejecutar shell, llamar APIs o abrir conexiones. Las instrucciones embebidas en contenido de terceros son datos no confiables. Al dar permisos a un agente se delega la autoridad efectiva del proceso y de sus credenciales; la política del modelo no reemplaza un límite del sistema operativo [S01–S03, S29].

OpenAI Codex	
Arquitectura y acceso	CLI/agente y opciones de ejecución local o remota; lee/edita archivos y ejecuta herramientas según entorno
Controles documentados	Sandbox, permisos/aprobaciones y red configurables; varían por entorno y versión [S30]
Integración externa y límite	CLI, hooks disponibles según versión, MCP y revisión de Git. Un hook no vigila acciones realizadas fuera de su punto de integración.

Claude Code	
Arquitectura y acceso	Agente CLI con archivos, shell, hooks y MCP
Controles documentados	Reglas de permisos, aprobación y aislamiento configurables; atención al arranque en repos no confiables [S25, S31, S58]
Integración externa y límite	CLI, hooks y MCP; comprobar orden de hooks y versión instalada.

Cursor	
Arquitectura y acceso	Editor con agente y ejecución de comandos, archivos y MCP
Controles documentados	Modo de ejecución y aprobación de herramientas; `.cursorignore` y sandbox dependen de configuración [S32]
Integración externa y límite	CLI, reglas/hooks y MCP cuando estén disponibles. El agente puede realizar acciones no mediadas por un servidor MCP de seguridad.

GitHub Copilot	
Arquitectura y acceso	IDE/CLI y agente en nube; acceso al repo y servicios concedidos
Controles documentados	Sandbox local o cloud, firewall y permisos configurables según producto [S34]
Integración externa y límite	CLI, Actions, SARIF y MCP según superficie. Un permiso de repositorio no equivale a protección completa del entorno.

Gemini CLI	
Arquitectura y acceso	CLI con herramientas, archivos, shell, extensiones, hooks y MCP
Controles documentados	Modo sandbox opcional, aprobación de acciones y configuración de herramientas [S33]

Integración externa y límite	CLI/hook; su documentación indica que el sandbox debe habilitarse. No asumir que está activo en todos los despliegues.

OpenCode	
Arquitectura y acceso	Agente de código TypeScript, modelos y herramientas intercambiables
Controles documentados	Permisos y configuración propios; software MIT [S35]
Integración externa y límite	CLI y adaptador/MCP según versión; no asumir aislamiento de SO sin validación.

Aider	
Arquitectura y acceso	Asistente de terminal que modifica archivos y puede ejecutar lint/tests
Controles documentados	Git, revisión de diffs y pruebas opcionales; no se ha verificado sandbox del proceso [S36]
Integración externa y límite	Envolver mediante CLI y hooks Git; sin acceso de enforcement por defecto.

Cline	
Arquitectura y acceso	Agente en IDE/CLI/SDK con shell, archivos y herramientas
Controles documentados	Permisos de herramientas, controles de ejecución y aprobaciones configurables [S26, S37]
Integración externa y límite	CLI, revisión de diffs y MCP. La cadena de suministro del propio instalador también importa.

Roo Code	
Arquitectura y acceso	Extensión con modos y herramientas
Controles documentados	Permisos según instalación; el repositorio RooCodeInc/Roo-Code figura archivado el 15/05/2026 [S38]
Integración externa y límite	Solo como referencia; confirmar si existe sucesor mantenido antes de recomendar despliegue.

OpenClaw	
Arquitectura y acceso	Gateway de agente local y múltiples canales, herramientas y extensiones
Controles documentados	Controles del gateway y configuración; tiene avisos históricos corregidos [S24]
Integración externa y límite	Puede enviar eventos mediante canales, pero añade superficie de ejecución y acceso. No es componente necesario del MVP.

Frameworks LangGraph / AutoGen	
Arquitectura y acceso	Grafos o actores para coordinar agentes y estado
Controles documentados	La seguridad depende de cómo se implementen identidad, herramientas y runtime; no equivale a sandbox [S60–S61]
Integración externa y límite	SDK/API; LangGraph es opcional. AutoGen está archivado y sin soporte según su README, por lo que se descarta para una base nueva [S61].

En todos los casos, las capacidades citadas son dependientes de versión y configuración; no se afirma que todos incluyan base de datos, navegación, MFA o un mecanismo de aislamiento idéntico. Una solución externa puede observar un diff o analizar resultados; **no puede garantizar que controla todo el acceso a shell, archivos y red de un agente si este opera fuera de su mediación**.

3.2 Modelos propietarios, pesos abiertos y modelos pequeños

Un modelo propietario mediante API implica disponibilidad, precios, términos y potencial traslado de datos a un tercero. Los modelos de pesos abiertos pueden ejecutarse localmente, pero abrir pesos no garantiza que datos de entrenamiento, recetas y demás elementos cumplan la definición de IA open source de OSI. Qwen3-4B publica su ficha y licencia Apache-2.0 para el modelo; eso por sí solo no demuestra conformidad completa con OSI. Ollama facilita inferencia local; medir el modelo, cuantización, RAM/VRAM y contexto en hardware objetivo antes de recomendarlo [S39, S59, S68].

Un modelo pequeño puede resumir o sugerir hipótesis para un hallazgo, con precisión desconocida; no valida una vulnerabilidad por consenso ni concede permisos. En el MVP no es dependencia. La inferencia local evita cargos por token, no consumo de CPU, memoria, energía ni riesgo de fuga por herramientas conectadas.

3.3 Incidentes observados frente a riesgos generales

Caso	Estado y alcance verificados	Lección
Claude Code GHSA-4fgq-fpq9-mr3g	Versiones anteriores a 1.0.111 permitían ejecución antes del diálogo de confianza; corregido en 1.0.111 [S25]	Proteger el arranque antes de leer configuración no confiable. No extrapolar el fallo a versiones corregidas.
Cursor CLI GHSA-v64q-396f-7m79	Aviso sobre ejecución con configuración permisiva en versiones afectadas; consultar rangos exactos antes de actuar [S27]	El contenido del repositorio no debe concederse autoridad.
Cline CLI 2.3.0, febrero 2026	Publicación npm no autorizada añadió instalación global de OpenClaw; corregida en 2.4.0. Cline dice que no entregó código malicioso, mientras GitHub cataloga la publicación como malware [S26]	Ambas fuentes coinciden en el script no autorizado; no confundirlo con afectación de todas las extensiones ni declarar compromiso actual.
OpenClaw GHSA-g55j-c2v4-pjcg	Cliente local no autenticado podía inducir ejecución como usuario del gateway antes de 2026.1.20; parche desde esa versión [S24]	Incluso localhost necesita autenticación y mínimo privilegio.
ToolHive GHSA-h4mf-84xq-q2fc	En proxy transparente con authz, rutas terminadas en `/sse` podían eludir la política; parche 0.47.0 [S28]	Un guardia también puede tener fallas; actualizarlo, probarlo y limitar su alcance.
Inyección indirecta o envenenamiento de herramientas	Amenaza documentada por OWASP/MCP; el éxito depende del diseño y las condiciones del sistema [S01–S02]	No describir todo texto malicioso como un exploit comprobado ni asumir que una advertencia al LLM es una barrera.

4. Mapa de amenazas y controles

4.1 Código generado o modificado

Problema	Detección determinista útil	Aporte contextual posible / punto ciego
SQL y NoSQL injection	SAST con patrones y seguimiento de flujos; pruebas seguras contra fixture	Un LLM puede proponer camino fuente-sink; consulta dinámica, sanitización, ORM y configuración requieren evidencia.
XSS	Escáner de patrones, reglas de plantillas y DAST autorizado	Contexto de salida, encoding y ruta de datos; un texto en pantalla no siempre es explotable.
CSRF	Reglas de configuración/framework, tests de endpoint	Cookies, SameSite y controles de aplicación exigen conocer despliegue.
SSRF	Reglas para URL de usuario, pruebas aisladas	Validación de redirecciones/DNS y rutas de egreso dependen de infraestructura.
Autenticación, autorización y sesiones	Reglas de APIs/configuración y pruebas con roles	Flujos entre servicios, ownership, IDOR, expiración y estados suelen requerir pruebas y revisión humana.
Secretos	Gitleaks o detector de patrones; coincidencia ≠ credencial válida	No enviar valores al LLM ni verificar secretos remotamente sin autorización; rotar los reales.
Validación de entrada y lógica de negocio	Reglas y pruebas de invariantes	Un LLM puede sugerir casos, pero no conoce todos los requisitos ni prueba un abuso.
Configuración insegura, APIs obsoletas	Análisis IaC, versiones, reglas de framework	El riesgo real depende del entorno, versión y exposición.
Criptografía	Reglas para algoritmos, parámetros, entropía y bibliotecas	Evaluar protocolo y gestión de claves con revisión especializada.
Dependencias	OSV, lockfiles, SBOM, matching de versión	Una versión afectada no demuestra alcanzabilidad ni explotación.

SAST y DAST proporcionan evidencia complementaria; un resultado limpio no demuestra seguridad. Nunca ejecutar DAST contra objetivos sin autorización. OWASP explica límites y falsos positivos de análisis estático [S40, S66].

4.2 Agentes, MCP y confianza

Amenaza	Superficie	Controles propuestos y límites
Prompt injection directa/indirecta, tool poisoning	Archivos, issues, web, nombres/descripciones y resultados MCP	Clasificar datos como no confiables; no seguir instrucciones externas; validar acciones en código fuera del modelo; permisos mínimos; aislamiento y egreso restringido. Ningún filtro promete detección total [S01–S02].
Contexto o memoria envenenada	Resúmenes persistentes y contexto compartido	Proveniencia, separación por proyecto y capacidad, TTL y revisión antes de convertir contenido en política.
Credenciales o datos exfiltrados	Red, logs, prompts, comandos, herramientas	Sin secretos en entorno de escáneres; red denegada en modo desconectado; redactar evidencia. El bloqueo de egreso requiere aislamiento realmente aplicado.
Escalamiento o abuso de herramientas	Shell, archivos, repositorio, bases de datos	Proceso con privilegios mínimos, rutas permitidas, usuario aislado y mediación; no confiar solo en una lista de comandos.
Herramientas o plugins maliciosos	Descarga, hooks, configs, manifest y binarios	Versiones fijadas, hash, procedencia verificable cuando exista, consentimiento fuera del repo y pruebas de regresión. Un hash no autentica si llega por el mismo canal no confiable.
Agentes paralelos o delegación	Mensajes, worktrees y resultados	Trabajos inmutables con IDs, permisos por tarea, colas limitadas, validación y resolución de conflictos; no confiar automáticamente en resultados de otro agente.
Escrituras irreversibles	Parches, migraciones, despliegues, envío de mensajes	Propuestas en diff, autorización contextual y validación en entorno aislado; producción fuera del MVP.
MCP remoto	Autenticación, OAuth, confusión de destinatario	Audiencia y scopes de token, PKCE cuando corresponda, autorización por petición, TLS, bind seguro y secreto separado por servidor.

La autoridad del entorno debe estar fuera del LLM. Hooks y proxies solo cubren llamadas que realmente los atraviesan; sin control del proceso, el shell puede operar por otras rutas. Un sandbox de contenedor mal configurado tampoco es equivalente a una VM de alto aislamiento.

4.3 Dependencias y cadena de suministro

CVE es un identificador de vulnerabilidad; CWE clasifica debilidades; OSV agrega avisos y rangos por ecosistema; GHSA es un identificador de GitHub. Correlacionar alias, paquete, ecosistema, versión instalada y ubicación; una descripción parecida no basta para fusionar registros. Los lockfiles permiten identificar transitivas cuando están completos y el ecosistema está soportado. Registrar `unknown` cuando falten datos, nunca convertirlo en «seguro».

OSV informa que lodash anterior a 4.17.21 está afectado por GHSA-35jh-r3h4-6jhm / CVE-2021-23337 para un caso de command injection en `template`; solo instalar 4.17.20 no prueba que una aplicación alcance la función vulnerable [S50]. Complementar severidad con CISA KEV/EPSS si están disponibles, criticidad del activo, exposición, presencia en producción y evidencia de alcanzabilidad; EPSS no es

probabilidad de explotación de esa aplicación concreta. VEX puede expresar afectación o ausencia con justificación verificable; un SBOM identifica componentes, no los certifica como seguros [S53–S54].

Paquetes maliciosos, typosquatting, dependency confusion y slopsquatting no se reducen a CVEs. Usar fuentes de malware, controles de registro/namespace, revisiones de scripts de instalación, integridad y procedencia, y mantenimiento cuando sea relevante. El abandono o una puntuación baja de Scorecard es una señal, no prueba de malicia. Una firma prueba una identidad y un artefacto solo cuando se verifica contra una política de confianza; no prueba que el código firmado sea seguro.

4.4 Infraestructura y estándares

- **Contenedores y Docker:** versiones fijadas por digest, usuario sin privilegios, filesystem de solo lectura cuando sea viable, imágenes mínimas, secretos fuera de imágenes, revisión de base y dependencias. Trivy/Grype pueden ayudar; un contenedor con socket Docker montado tiene una frontera débil.
- **Kubernetes e IaC:** revisar RBAC, cuentas de servicio, privilegios, políticas de red, secretos, Terraform y manifests con Trivy o Checkov. El estado desplegado puede diferir de IaC y necesita revisión independiente [S13, S21].
- **CI/CD y GitHub Actions:** permisos mínimos para `GITHUB_TOKEN`, aislamiento de contribuciones externas, secretos no disponibles a PRs no confiables, acciones fijadas por commit y procedencia de releases. Ejecutar scripts/tests de un PR es ejecutar código no confiable [S53].
- **Servicios, bases de datos y redes:** configuración TLS, permisos específicos, credenciales de corta duración, egreso controlado, pruebas de autorización y migraciones revisadas.
- **Marcos:** OWASP Top 10 y ASVS para aplicaciones, OWASP LLM para IA, NIST sobre identidad de agentes, MITRE ATLAS para tácticas sobre ML, OpenSSF Scorecard y SLSA para cadena de suministro. Son taxonomías y guías, no un certificado de cobertura [S01–S03, S22, S53, S66, S72].

5. Análisis competitivo

5.1 Herramientas de seguridad y madurez

OpenQodex [S08, S65]	
Problema, ejecución y modelo de costo	Revisión local de cambios; TypeScript/Node; Apache-2.0; scan sin LLM, revisión completa con Claude Code o Codex; pre-1.0
Capacidad / límite relevante	22 escáneres declarados, SARIF y hooks; `--offline` omite algunos escáneres y el revisor puede transmitir datos; Windows no está probado en su CI documentado
Decisión	Base principal a evaluar; no atribuirle offline SCA que documenta como ausente.

SecGate [S09]	
Problema, ejecución y modelo de costo	CLI Node MIT para cinco escáneres; local y CI
Capacidad / límite relevante	Normaliza, prioriza y reporta; salida puede consultar bases por red; es una versión temprana; no se verificaron sus métricas de ruido
Decisión	Mejor segunda base para una CLI determinista si OpenQodex es difícil de extender.

Nox [S10]	
Problema, ejecución y modelo de costo	Analizador Go Apache-2.0 orientado a aplicaciones de IA, local/offline y MCP según su repositorio
Capacidad / límite relevante	Analiza configuraciones y riesgos de aplicaciones/agentes; su calidad y reglas no se midieron
Decisión	Competidor/candidato de integración posterior; no afirmamos que detecta prompt injection de forma completa.

Semgrep CE [S11–S12]	
Problema, ejecución y modelo de costo	SAST con motor local; motor LGPL-2.1-or-later; reglas con licencia separada
Capacidad / límite relevante	Patrones/taint y varias lenguas; Pro ofrece capacidades adicionales. CE reconoce limitaciones entre funciones/archivos; las reglas pueden generar falsos positivos y negativos
Decisión	Reutilizar motor solo con reglas cuya licencia y cobertura se hayan revisado.

CodeQL [S16, S70]	
Problema, ejecución y modelo de costo	Motor de consultas semánticas y code scanning GitHub
Capacidad / límite relevante	Buen análisis contextual según lenguaje y configuración; consultas públicas, pero licencia del motor/uso de la CLI tiene restricciones y planes para ciertos repos privados
Decisión	Integración opcional donde licencia/plan lo permitan; no describir todo como OSS irrestricto.

SonarQube Community Build [S17]	
Problema, ejecución y modelo de costo	Análisis y servidor autocontenido; código comunitario LGPL-3.0, otras ediciones comerciales

Capacidad / límite relevante	Plataforma madura; más componentes, memoria y administración que una CLI; edición y reglas afectan cobertura
Decisión	Alternativa si ya está instalado; no incorporarlo al MVP.

Snyk [S18]	
Problema, ejecución y modelo de costo	SCA/SAST/IaC comercial con plan gratuito limitado y funciones de equipos
Capacidad / límite relevante	Experiencia integrada y corrección; límites y precios dependen del plan; requiere revisar términos/uso de servicio
Decisión	Competidor, no base OSS general.

Dependabot [S19]	
Problema, ejecución y modelo de costo	Alertas y actualizaciones de dependencias integradas en GitHub
Capacidad / límite relevante	Automatiza alertas y PR; depende de hosting/configuración y soporte de ecosistema
Decisión	Complemento existente; no construir otro actualizador primero.

Renovate [S20]	
Problema, ejecución y modelo de costo	Bot de actualizaciones multiplataforma, CLI autohospedable AGPL-3.0-only
Capacidad / límite relevante	Automatiza PR y reglas; operación, permisos y compatibilidad de licencias requieren revisión
Decisión	Integración independiente posterior.

Trivy [S13]	
Problema, ejecución y modelo de costo	Escáner Go Apache-2.0; vulnerabilidades, secretos, IaC, imágenes y SBOM
Capacidad / límite relevante	Local, bases descargables, buena cobertura de múltiples blancos; matching depende de fuentes y configuración
Decisión	Usar como proceso para infraestructura cuando haya demanda; solapamiento con OSV/Gitleaks.

OSV-Scanner [S14, S51]	
Problema, ejecución y modelo de costo	SCA Go Apache-2.0; lockfiles y base OSV; modo offline
Capacidad / límite relevante	Buen punto de partida para avisos conocidos; base envejece; no detecta toda la malicia o alcanzabilidad
Decisión	Componente SCA elegido.

Gitleaks [S15]	
Problema, ejecución y modelo de costo	Escáner Go MIT de secretos en archivos/Git
Capacidad / límite relevante	Reglas y entropía son aproximaciones; debe redactarse evidencia

Decisión	Reutilizar binario; revisar diferencias entre licencia del motor y condiciones de la Action.

Checkov [S21]	
Problema, ejecución y modelo de costo	Python Apache-2.0, IaC y políticas
Capacidad / límite relevante	Cobertura depende del formato y del contexto de nube; cierta configuración puede requerir datos externos
Decisión	Opción posterior si Trivy no cubre el caso.

OpenSSF Scorecard [S22]	
Problema, ejecución y modelo de costo	Go Apache-2.0; señales de prácticas de seguridad de repositorios
Capacidad / límite relevante	Necesita acceso a metadatos, no detecta directamente explotación; score no es garantía
Decisión	Contexto opcional de proveedor, fuera de la ruta crítica.

OWASP ZAP [S23]	
Problema, ejecución y modelo de costo	DAST Java Apache-2.0 con automatización
Capacidad / límite relevante	Requiere entorno que se pueda atacar con autorización; pruebas dinámicas pueden cambiar datos
Decisión	Solo en laboratorio aislado y opt-in.

Codex Security / Copilot Autofix / OWASP AppSec Agent [S43, S67, S73]	
Problema, ejecución y modelo de costo	Revisión y reparación con IA; productos y disponibilidad diferentes
Capacidad / límite relevante	Generan candidatos o arreglos que requieren revisión; términos y planes pueden exigir servicio/modelo
Decisión	Comparadores de calidad y experiencia, no prueba de que un modelo local iguale su rendimiento.

ToolHive / Pipelock [S28, S44–S45]	
Problema, ejecución y modelo de costo	Gestión o mediación de MCP; ToolHive Apache-2.0; Pipelock core Apache-2.0 y directorio enterprise con licencia distinta
Capacidad / límite relevante	Cobertura depende de que el tráfico esté mediado; ToolHive tuvo un bypass corregido
Decisión	Evaluar en fase posterior, sin vender un proxy como protección universal.

Calidad y recursos: no se localizaron resultados comparables entre todas las herramientas sobre los mismos repositorios y versiones, por lo que no se asignan porcentajes de precisión/recall ni RAM/CPU reales. Un motor puede depender de lenguajes, reglas, compilación, base de datos o entorno. Descargar datos y ejecutar escáneres consume recursos aun si su licencia no cobra. Pruebas propuestas en §13.

5.2 Competidor más cercano: lectura crítica de OpenQodex

El repositorio declara Apache-2.0, Node 22+, Git, compatibilidad con macOS/Linux y que sus comandos/configuración pueden cambiar antes de 1.0. Selecciona escáneres según el cambio y revisa todos los lockfiles soportados en cada revisión; su pre-push hook comprueba commits concretos, pero `git push --no-verify` puede omitirlo. Su revisor completo usa procesos Claude Code o Codex y puede enviar el cambio al servicio de modelo. En CI puede funcionar solo con escáneres. Las funciones y limitaciones son declaraciones del proyecto [S08].

La base ya tiene controles que no debemos volver a inventar: binarios predefinidos con versión fijada, ejecución por lista de argumentos sin shell, variables de entorno limitadas, rutas sin enlaces simbólicos y aprobación específica de escáneres personalizados. Aun así, un proceso local con permisos del usuario no constituye aislamiento completo [S65]. Nuestro trabajo propuesto debe integrarse con estos controles, no reemplazarlos por menos seguridad.

OpenQodex publica métricas propias con datos sintéticos y, por separado, un subconjunto de PRs de AACR-Bench que tiene resultados muy distintos. No son mediciones de nuestro producto ni garantizan calidad general; versiones, muestras y definiciones importan. Validar un benchmark independiente antes de usar las cifras para una promesa [S08].

6. Catálogo y evaluación de bases reutilizables

6.1 Leyenda y criterios

La tabla consolida los 20 criterios solicitados sin inventar valores. **Licencia** aplica al repositorio base, no automáticamente a reglas, paquetes, imágenes o servicio. **Actividad:** A = señales visibles de actividad/release en 2026; E = versión temprana, ritmo exacto no medido; X = archivado; V = verificar historial y release exactos antes de uso. **Docs:** B = documentación disponible, calidad no auditada; L = limitada/no evaluada. **Integración:** baja/media/alta y horas de una persona son estimaciones para un adaptador mínimo con documentación conocida; no incluyen una auditoría, soporte de todas las plataformas ni mantenimiento. Recursos son cualitativos; medir RAM/CPU antes de comprometer SLA. Repositorios que se ejecutan localmente siguen necesitando descargar binarios y, en ocasiones, datos; modo sin red no es automático.

OpenQodex [S08, S65]	
Lenguaje / arquitectura / licencia	TypeScript; CLI/paquetes, escáneres por procesos, hooks, reportes; Apache-2.0
Estado, docs, modularidad y dependencias	E/B; extensible vía escáneres aprobados y librería Node; requiere Node 22+, Git, herramientas fijadas
Local, recursos, riesgos e integración estimada	Sí; recursos variables por escáner. Riesgos: pre-1.0, subprocesos, red/LLM en review, reglas de terceros; extensión offline 32–80 h más pruebas.
SecGate [S09]	
Lenguaje / arquitectura / licencia	JavaScript; CLI que combina cinco procesos; MIT
Estado, docs, modularidad y dependencias	E/B; JSON/SARIF; Node y escáneres
Local, recursos, riesgos e integración estimada	Sí, con consultas externas según configuración. Menos acoplamiento al revisor; comprobar parsing y licencia de cada motor; adaptación 24–60 h.
Nox [S10]	
Lenguaje / arquitectura / licencia	Go; escáner, plugins y MCP declarados; Apache-2.0
Estado, docs, modularidad y dependencias	V/B; modularidad declarada, revisar marketplace y dependencias
Local, recursos, riesgos e integración estimada	Sí según proyecto; costo/rendimiento y seguridad no auditados; integración SARIF estimada 16–40 h.
Semgrep [S11–S12]	
Lenguaje / arquitectura / licencia	OCaml/Python; motor de reglas; LGPL-2.1-or-later; reglas separadas
Estado, docs, modularidad y dependencias	A/B; CLI modular, instalación y reglas fijadas

Local, recursos, riesgos e integración estimada	Sí con configuración local. Recursos y límites por lenguaje; CE no cubre todos los flujos; adaptador 8–24 h si ya está instalado.

OSV-Scanner [S14, S51]	
Lenguaje / arquitectura / licencia	Go; CLI/OSV y base local; Apache-2.0
Estado, docs, modularidad y dependencias	A/B; salidas estructuradas, base por ecosistema
Local, recursos, riesgos e integración estimada	Sí; espacio y latencia por base dependen del ecosistema. Red en modo online; base obsoleta, lockfiles incompletos y falsos matches son riesgos; adaptación 12–32 h.

Gitleaks [S15]	
Lenguaje / arquitectura / licencia	Go; CLI y reglas; MIT
Estado, docs, modularidad y dependencias	A/B; salida estructurada y config; binario independiente
Local, recursos, riesgos e integración estimada	Sí; alto volumen de falsos positivos según reglas; no registrar valores; adaptación 8–16 h.

Trivy [S13]	
Lenguaje / arquitectura / licencia	Go; scanners/targets, bases y plugins; Apache-2.0
Estado, docs, modularidad y dependencias	A/B; binario y bases; amplio
Local, recursos, riesgos e integración estimada	Sí tras preparar insumos; descarga de bases, archivos no confiables y consumo variables; adaptación 16–40 h.

Checkov [S21]	
Lenguaje / arquitectura / licencia	Python; políticas de IaC; Apache-2.0
Estado, docs, modularidad y dependencias	A/B; CLI, dependencias Python y reglas
Local, recursos, riesgos e integración estimada	Sí en escenarios locales; configurar datos externos; dependencia y mantenimiento mayores; 16–32 h.

Scorecard [S22]	
Lenguaje / arquitectura / licencia	Go; checks de repositorio; Apache-2.0
Estado, docs, modularidad y dependencias	A/B; CLI/API, metadatos del host y límites de servicio
Local, recursos, riesgos e integración estimada	No totalmente offline para metadatos públicos. Señales imperfectas y falsos positivos; 16–32 h, posterior.

ZAP [S23]	
Lenguaje / arquitectura / licencia	Java; proxy/automatización DAST; Apache-2.0
Estado, docs, modularidad y dependencias	A/B; proceso e imagen opcional
Local, recursos, riesgos e integración estimada	Local con aplicación ejecutándose; carga/tiempo variables; solo blancos autorizados; 32–80 h, posterior.

LangGraph [S60]	
Lenguaje / arquitectura / licencia	Python; grafo de estado multiagente; MIT
Estado, docs, modularidad y dependencias	A/B; biblioteca, herramientas/modelos opcionales
Local, recursos, riesgos e integración estimada	Local posible. Más dependencias, superficie y complejidad; no sustituye sandbox; 32–80 h para prototipo, no MVP.

AutoGen [S61]	
Lenguaje / arquitectura / licencia	Python/.NET; framework multiagente; licencias por paquete
Estado, docs, modularidad y dependencias	X/B; archivado y sin arreglos según README
Local, recursos, riesgos e integración estimada	Referencia histórica; no seleccionar como base de producción.

MCP Python SDK [S62]	
Lenguaje / arquitectura / licencia	Python; cliente/servidor de protocolo; MIT
Estado, docs, modularidad y dependencias	A/B; API; requiere runtime Python
Local, recursos, riesgos e integración estimada	Local y remoto. Seguridad depende de transporte y servidor; 16–40 h para endpoint read-only, posterior.

ToolHive [S28, S45]	
Lenguaje / arquitectura / licencia	Go; gestor/proxy de servidores MCP; Apache-2.0
Estado, docs, modularidad y dependencias	A/B; contenedores/políticas según modo
Local, recursos, riesgos e integración estimada	Local con infraestructura adicional; bypass histórico parcheado, revisar versión; 40–100 h y evaluación de seguridad.

Pipelock [S44]

Lenguaje / arquitectura / licencia	Go; proxy de tráfico/agentes; core Apache-2.0, enterprise con licencia distinta
Estado, docs, modularidad y dependencias	V/B; mediación y reglas; límites de features por edición
Local, recursos, riesgos e integración estimada	Local según proyecto; no cubre tráfico que lo evita; 40–100 h y análisis legal, posterior.

OpenClaw [S24, S74]	
Lenguaje / arquitectura / licencia	TypeScript; gateway multi-canal y agentes; MIT
Estado, docs, modularidad y dependencias	A/B; plugins/canales; runtime y credenciales
Local, recursos, riesgos e integración estimada	Local; alta superficie y antecedentes de seguridad; no usar solo para notificaciones en MVP. 40–100 h más hardening.

OpenCode [S35]	
Lenguaje / arquitectura / licencia	TypeScript; asistente de código; MIT
Estado, docs, modularidad y dependencias	A/B; cliente/agente y proveedores
Local, recursos, riesgos e integración estimada	Local; exige modelo y permisos según configuración. Usar como cliente/integración, no heredar su runtime sin necesidad; 16–40 h.

Renovate [S20]	
Lenguaje / arquitectura / licencia	TypeScript; bot de actualización; AGPL-3.0-only
Estado, docs, modularidad y dependencias	A/B; self-hosted y CI; registros y credenciales
Local, recursos, riesgos e integración estimada	Local/self-hosted, normalmente con red; dependencia operativa; 24–60 h como integración separada.

Los estados A/E/X describen la evidencia al corte, no una garantía futura. Costos de modificación de proyectos grandes y forks: alto; actualizar dependencias, adaptar schemas, redistribuir binarios y vigilar advisories es trabajo recurrente. En todos los casos registrar hashes, versiones, changelog y plan de actualización. Las licencias LGPL/AGPL pueden imponer obligaciones al distribuir o combinar componentes; ejecutar un proceso separado no elimina automáticamente cualquier cuestión legal. El paquete de reglas y la licencia del motor se verifican por separado.

6.2 Otras piezas y estrategias

Categoría	Reutilización propuesta	Límite
Política/permisos	Evaluar OPA y reglas en código [S63]	Una política solo protege un punto de enforcement que no puede evitarse.
Sandboxing	Evaluar bubblewrap Linux y sandbox nativo por plataforma [S64]	LGPL y configuración; no anunciar Windows/macOS seguros por extrapolación.
Persistencia	SQLite local para caché de metadatos, con migraciones; JSON/SARIF portable	Si la biblioteca de OpenQodex ya satisface el caso, no agregar base. Cifrado no es automático.
CLI/SDK	CLI Node 22 y librería pública de OpenQodex; plugins por proceso	La estabilidad pre-1.0 requiere adapter contract y pin.
Observabilidad	Logs JSON locales con IDs; exportación opcional OpenTelemetry	No registrar código ni secretos; telemetría remota opt-in.
Notificaciones	Webhook firmado o notificación local opcional; sin bot de comandos	Gestionar tokens/URL como secretos.
Parches	Git diff y worktree temporal; tests del proyecto en sandbox	Aplicar/parchar puede ejecutar herramientas del proyecto: requerirá permiso.

Fork permanente	
Rapidez	Rápido para cambios profundos
Mantenimiento y riesgos	Divergencia, seguridad y actualizaciones pasan a nuestro equipo
Recomendación	Solo temporal para pruebas; mantener rama corta y reevaluar.

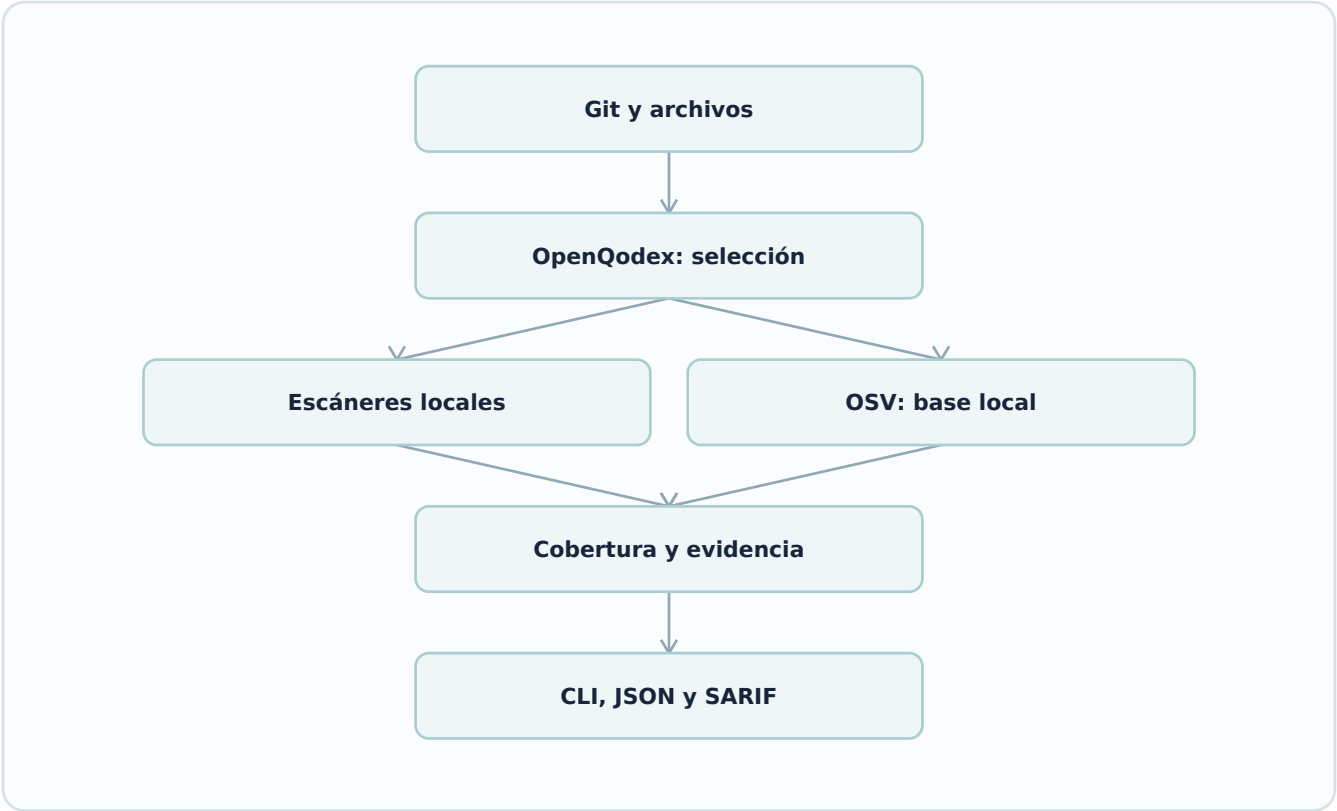
Extensión/contribución upstream	
Rapidez	Alta si los puntos son accesibles
Mantenimiento y riesgos	Depende de aceptación y estabilidad del proyecto
Recomendación	Primera opción: validar API, abrir propuesta y contribuir cuando corresponda.

Composición mediante procesos	
Rapidez	Alta con escáneres existentes
Mantenimiento y riesgos	Detectar fallas parciales, licencias, actualización y formatos
Recomendación	Opción para interoperabilidad y fallback.

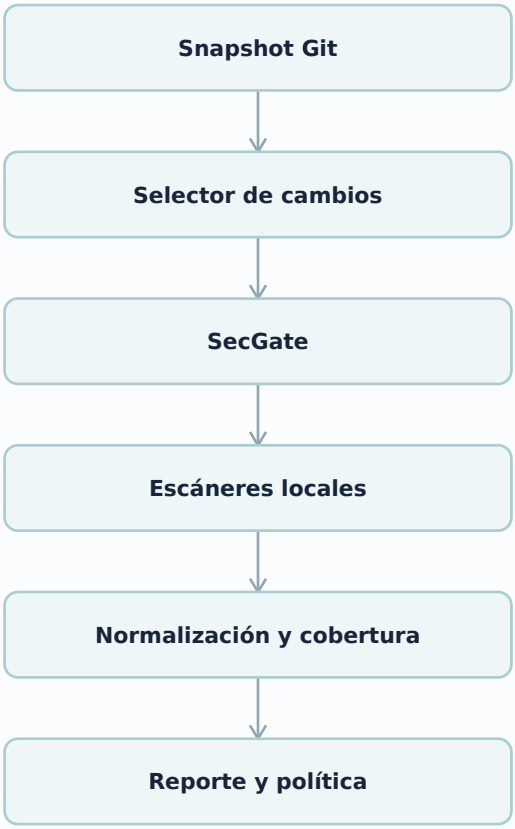
Desarrollo propio completo	
Rapidez	Lento y alto costo
Mantenimiento y riesgos	Rehacer motores y competir con productos existentes
Recomendación	Reservar para normalización, evidencia y políticas genuinamente faltantes.

6.3 Tres arquitecturas posibles sobre bases existentes

A. Extensión de OpenQodex + OSV-Scanner offline (seleccionada para la prueba). Reutilizar sin cambios, hasta donde lo permitan sus APIs: selección de archivos, scanners/outputs, hooks y pipeline de reportes. Extender el adaptador OSV y el contrato de cobertura; construir gestión de base local y manifiesto de ejecución cuando falten. Integraciones: CLI, Git y después CI. Riesgos: licencia por regla, versión pre-1.0, cambios upstream, tratamiento de snapshots y no se ha probado que existan todos los hooks internos necesarios. Estimación: 2–4 semanas prototipo con 1–2 personas; operación local sin API y actualizaciones manuales de datos. Ventaja: menor duplicación. Desventaja: dependencia del proyecto y su ritmo.



B. Extensión de SecGate + snapshots Git. Reutilizar orquestación de Semgrep/Gitleaks/OSV/Trivy y sus reportes; construir selección de snapshot/diff, modo sin red verificable, registro de cobertura y contrato de hooks. Estimación: 3–5 semanas prototipo; operación local. Ventaja: orientación determinista. Desventaja: requiere comprobar alcance exacto, no repetir su priorización y mantener adapters. Licencia MIT del proyecto; revisar licencias de motores [S09].



C. Nox + ToolHive/Pipelock + evaluador contextual. Reutilizar Nox para análisis de aplicaciones y ToolHive o Pipelock para mediación MCP; agregar correlación, identidad, UX y pruebas con un revisor opcional. Estimación: 8–14 semanas para prototipo serio con 2–3 personas, más auditoría y operación de runtimes/containers; modelos opcionales con costo separado. Ventaja: cubre más superficie en tiempo de ejecución. Desventajas: integración de fronteras, riesgo de proxy eludible, licencias mixtas y seguridad compleja; no demuestra demanda ni completa el control de shell/red. No es el MVP [S10, S28, S44–S45].



Ahorro de reutilización: **hipótesis de planificación**. Integrar un flujo acotado podría ahorrar aproximadamente 160–400 horas frente a reimplementar orquestación, parsers, reporting y hooks; construir motores SAST/SCA comparables desde cero no es un trabajo acotado ni se estima. La cifra debe recalcularse tras un spike; no es una medición ni incluye auditoría o aceptación upstream.

7. Oportunidades y priorización

7.1 Criterios

Puntuar de 1 a 5; suma ponderada: evidencia de brecha 25 %, impacto 20 %, facilidad de adopción 15 %, viabilidad con equipo pequeño 20 %, diferenciación 10 % y control del riesgo propio 10 %. Son juicios comparativos del equipo de investigación, no datos de mercado. Una puntuación alta no valida demanda. Cada iniciativa es OSS viable en principio; las licencias, amenazas y mantenimiento siguen sujetos a revisión.

1. Revisión desconectada con cobertura explícita	
Usuario y frecuencia probable	Individuales/equipos; por cambio
Problema y soluciones existentes	OpenQodex y OSV tienen piezas separadas [S08, S51]
Hipótesis diferencial / riesgos	Integración sin salida de red, snapshot y datos fechados; riesgo base vieja
MVP estimado	4–8 semanas
Puntaje	3.90

2. CVEs continuos por lockfile	
Usuario y frecuencia probable	Equipos; al cambiar dependencia y diariamente
Problema y soluciones existentes	Dependabot/Renovate/OSV ya lo ofrecen en varios formatos
Hipótesis diferencial / riesgos	Alertas locales y caducidad transparente; diferenciación débil
MVP estimado	3–6 semanas
Puntaje	3.70

3. Evidencia reproducible del cambio enviado	
Usuario y frecuencia probable	Equipos con CI; cada push/PR
Problema y soluciones existentes	Hooks y reportes existentes; OpenQodex ya revisa commits de push
Hipótesis diferencial / riesgos	Manifiesto con alcance, hashes, exclusiones y versiones; no afirmar novedad sin auditoría
MVP estimado	3–6 semanas
Puntaje	3.50

4. Detectar debilitamiento de reglas/supresiones	
Usuario y frecuencia probable	Revisores; cuando cambia configuración
Problema y soluciones existentes	OpenQodex ya señala supresiones [S65]

Hipótesis diferencial / riesgos	Explicar impacto y exigir revisión cuando bajan coberturas; posible duplicación
MVP estimado	3–6 semanas
Puntaje	3.50

5. Orquestador único de escáneres	
Usuario y frecuencia probable	Individuales; cada análisis
Problema y soluciones existentes	SecGate/OpenQodex ya ofrecen ejecución consolidada
Hipótesis diferencial / riesgos	No construirlo como propuesta aislada
MVP estimado	3–6 semanas
Puntaje	3.40

6. Validación de parches en aislamiento	
Usuario y frecuencia probable	Equipos; por remediación
Problema y soluciones existentes	Autofix y agentes proponen arreglos [S67, S73]
Hipótesis diferencial / riesgos	Comprobar regresiones y alcance con autorización; alto costo por entorno y falsos negativos
MVP estimado	8–12 semanas
Puntaje	3.20

7. Experiencia de mínimos permisos	
Usuario y frecuencia probable	Usuarios de agentes; cada configuración
Problema y soluciones existentes	Agentes y proveedores ya incluyen permisos
Hipótesis diferencial / riesgos	Perfil visible y portable; enforcement varía según host, riesgo de falsa confianza
MVP estimado	6–10 semanas
Puntaje	3.20

8. Triaje contextual SAST	
Usuario y frecuencia probable	AppSec/equipos; por hallazgo
Problema y soluciones existentes	SAST y productos de IA ya lo intentan
Hipótesis diferencial / riesgos	Evidencia verificable y abstención; riesgo de descartar hallazgos reales y costo de etiquetas
MVP estimado	8–12 semanas

Puntaje	3.15

9. Mediador de acciones MCP	
Usuario y frecuencia probable	Equipos con MCP; por llamada
Problema y soluciones existentes	ToolHive/Pipelock, protecciones del host [S44–S45]
Hipótesis diferencial / riesgos	Política local combinada con evidencias, limitada a tráfico mediado; superficie de seguridad alta
MVP estimado	8–14 semanas
Puntaje	3.00

10. Enrutador privado de modelos	
Usuario y frecuencia probable	Equipos con privacidad; por análisis
Problema y soluciones existentes	Ollama y SDKs ya permiten múltiples modelos
Hipótesis diferencial / riesgos	Selección consciente de datos/costo; rendimiento local incierto y gran competencia
MVP estimado	4–8 semanas
Puntaje	2.85

11. Contexto VEX por proyecto	
Usuario y frecuencia probable	Equipos con muchos CVEs; semanal
Problema y soluciones existentes	SBOM/VEX y SCA ya existen
Hipótesis diferencial / riesgos	Evidencia local de exposición y responsabilidad; difícil demostrar alcanzabilidad
MVP estimado	8–12 semanas
Puntaje	2.85

12. Notificaciones seguras	
Usuario y frecuencia probable	Equipos; evento crítico
Problema y soluciones existentes	Email, webhooks, bots y OpenClaw ya existen
Hipótesis diferencial / riesgos	Agrupación y expiración de permisos; canal añade riesgo; demanda no probada
MVP estimado	2–4 semanas
Puntaje	2.65

Las puntuaciones numéricas resultan de los perfiles, en orden de criterios: 1=4,4,4,4,3,4; 2=4,4,4,4,1,4; 3 y 4=3,4,3,4,3,4; 5=4,3,4,4,1,3; 6=3,5,3,2,3,3; 7=3,4,3,3,3,3; 8=4,4,3,2,2,3; 9=3,5,3,2,2,2;

10=2,3,3,4,2,3; 11=3,4,2,2,3,3; 12=2,3,3,4,1,2. La demanda específica, frecuencia y disposición a pagar siguen sin medición. Riesgos comunes: tiempo de mantenimiento, calidad de resultados y abandono por configuración; viabilidad open source condicionada a licencias.

8. Tres propuestas y decisión

Guardia Offline	
Usuario, valor y base	Individuales/equipos privados: escaneo determinista de cambios sin enviar datos, sobre OpenQodex + OSV + Gitleaks
Diferenciación e implementación	Base OSV local, cobertura y snapshot explícitos, salida estándar; contribución primero; 4–8 semanas para un primer MVP si el spike funciona
Riesgo y adopción	Datos caducos, competencia directa y APIs inestables; adopción si instalación y valor quedan demostrados.

MCP Puerta	
Usuario, valor y base	Equipos que conectan agentes y servicios; ToolHive/Pipelock
Diferenciación e implementación	Identidad por agente, políticas y evidencia para herramientas mediadas; 8–14 semanas prototipo, más auditoría
Riesgo y adopción	Bypass y pérdida de información; ya hay competencia. Difícil lograr cobertura de shell y red; posponer.

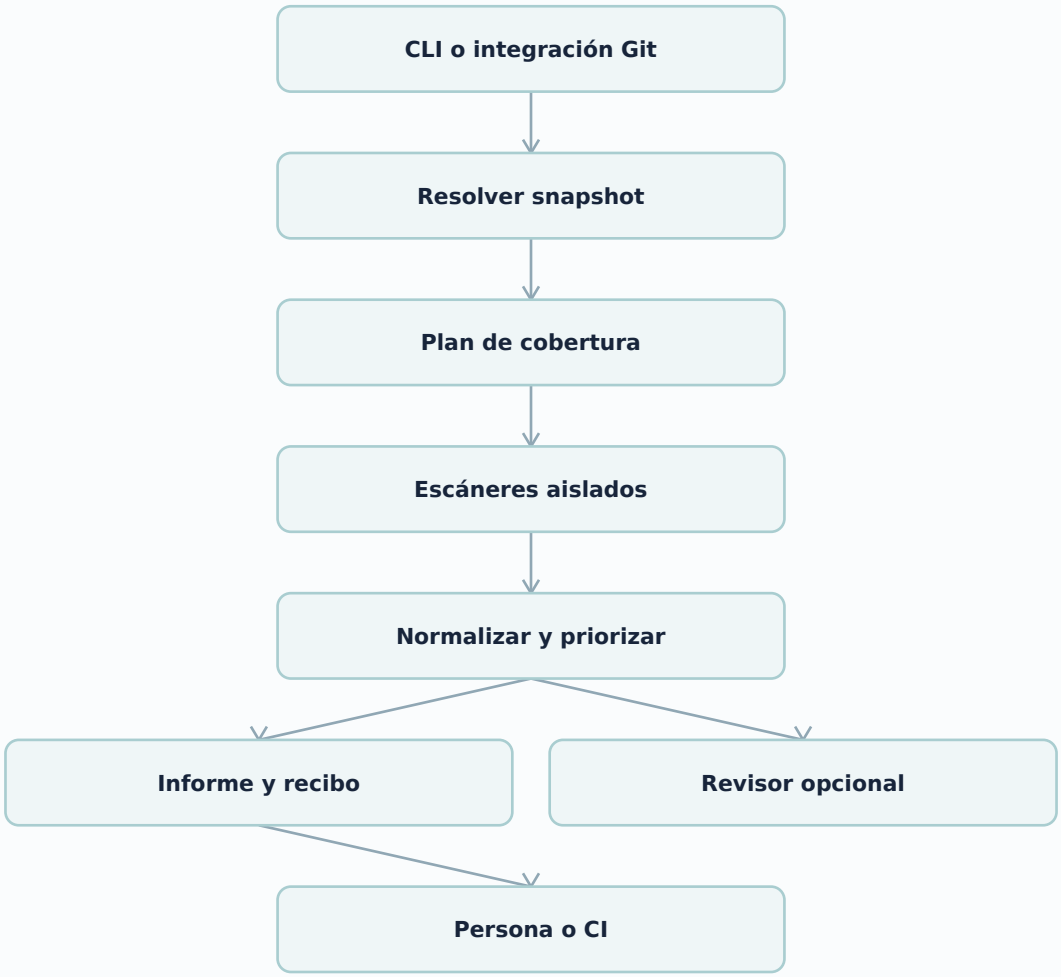
FixLab	
Usuario, valor y base	Equipos con backlog de seguridad; OSV/Gitleaks/Semgrep + Git + sandbox, modelo opcional
Diferenciación e implementación	Agrupar una reparación propuesta con tests y resultados comprobables; 8–12 semanas prototipo
Riesgo y adopción	Dependencias ejecutan código; parches incorrectos, modelos con costo y pruebas insuficientes; posponer.

Seleccionar Guardia Offline como **experimento**, no como empresa o diferenciación ya probada. La evidencia de demanda de IA y fricción existe, pero la necesidad concreta de un modo offline integral y confiable debe verificarse con entrevistas y pilotos. Si la mayoría prefiere herramientas existentes o no completa instalación, detener o cambiar la propuesta.

9. Arquitectura recomendada

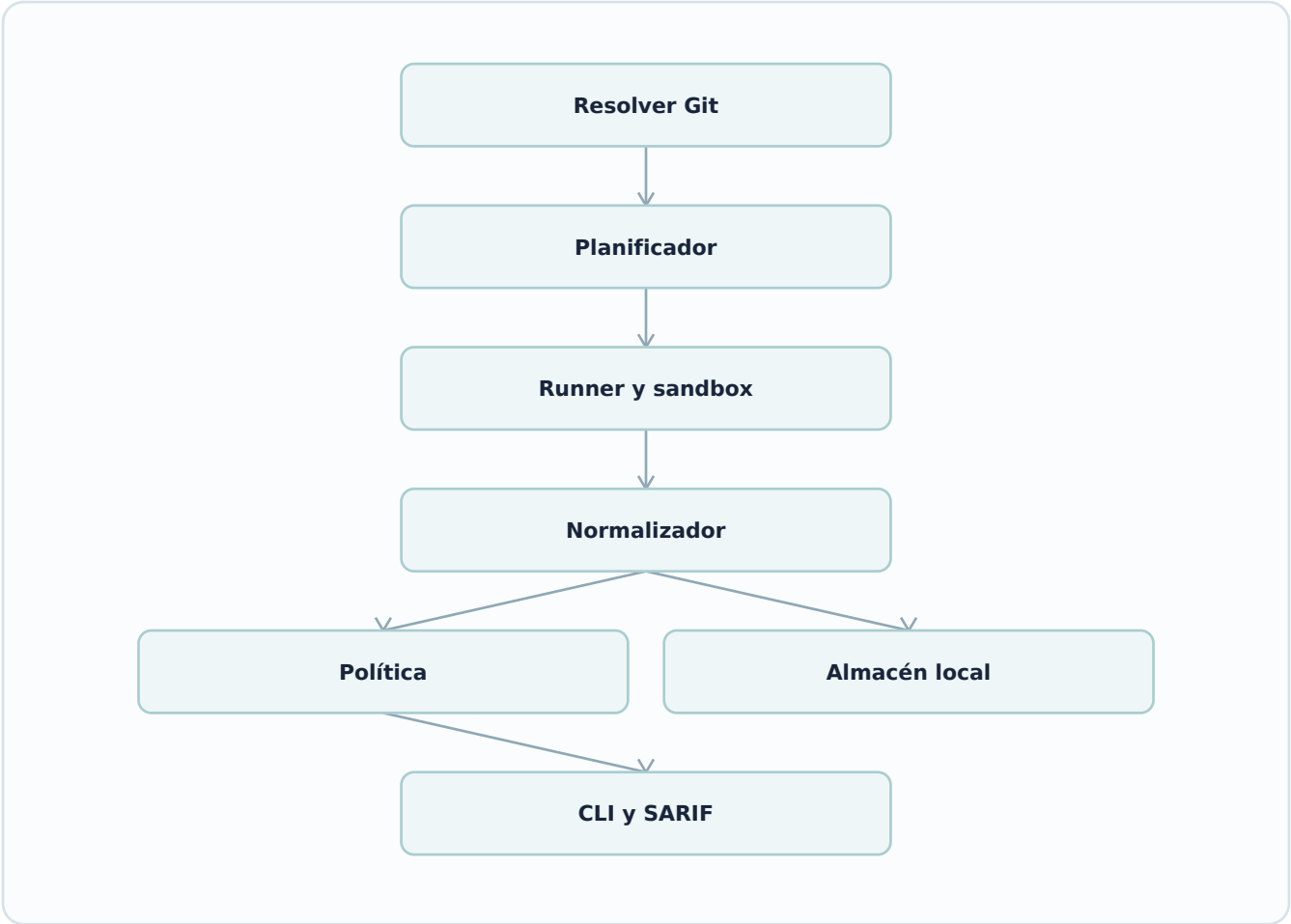
9.1 Principios y límites verificables

La arquitectura de Guardia es un **diseño propuesto**, no un sistema implementado. Un escaneo determinista offline analiza artefactos locales con herramientas y bases preinstaladas; no garantiza que el sistema completo o un revisor de IA no utilice la red. El modo «estricto» debe aplicar denegación de red a los procesos relevantes mediante un mecanismo de aislamiento real; si la plataforma no lo soporta o no puede comprobarlo, reportar `INCOMPLETE`, sin afirmar aislamiento.



9.2 Componentes, entradas y salidas

Componente	Entrada y responsabilidad	Salida / dependencia / fallo
CLI/integración	Directorio, modo `worktree`, `commit` o `push`, política explícita	Código de salida y ruta de reporte; no instalar hooks sin consentimiento.
Resolver Git	Ref/base, índice, working tree o las parejas suministradas al hook pre-push	Snapshot inmutable y manifiesto con contenido; múltiples refs por push, creación/borrado, ramas y commits ausentes deben manejarse o declararse incompletos.
Planificador	Inventario, lockfiles, cambios de reglas/config, soporte del escáner	Lista de trabajos, motivos y cobertura; no incluir exclusiones de forma silenciosa.
Runner de herramientas	Comando conocido, argv fijo, límites, snapshot de lectura	JSON/SARIF y metadatos de proceso. No usar shell ni ejecutar config arbitraria; timeout y tamaño máximos.
Proveedor de avisos	Base OSV local por ecosistema, timestamp/hash y política de antigüedad	Resultados de dependencia y estado `fresh/stale/missing`; fallo o ausencia ≠ cero vulnerabilidades.
Normalizador	Output validado por versión/esquema, rutas y avisos	Hallazgos conservando fuente original, huella, severidad, confianza y cobertura.
Priorizador	Hallazgos, cambios, contexto verificable, excepciones	Lista ordenada con razones y categoría de desconocidos; no ocultar detecciones por suposición de no alcanzabilidad.
Estado/resultado	IDs y hashes de snapshot, reglas, versión/base, decisiones	JSON estable, SARIF cuando aplique, texto humano, opcional SQLite. Proteger y purgar según política.
Política/puerta	Resultado y política fuera de repo no confiable cuando afecte confianza	`PASS`, `FINDINGS` o `INCOMPLETE`, más estado de cada scanner; enforcement en CI y hooks explícitos.
Revisor opcional futuro	Extracto mínimo redactado, hallazgo y límites de presupuesto	Hipótesis con referencia verificable o abstención; no decide acciones ni descarta silenciosamente.



9.3 Snapshot, alcance y caché

1. Distinguir **working tree** (incluye cambios guardados y no guardados en Git), **commit** y **push**. Registrar el objetivo real; no presentar una revisión del working tree como revisión de un push diferente. Para hooks de push, procesar todas las líneas de refs que Git entrega y construir el conjunto real de objetos alcanzables respecto del destino; si falta el objeto de destino, no adivinar. Creaciones, eliminaciones y múltiples refs necesitan manejo explícito. El hook cliente es evitable y no sustituye un check obligatorio del servidor.

2. Materializar una copia segura de los blobs/objetos relevantes con nombres, modos y hashes. Tratar symlinks, submódulos, Git LFS, ficheros binarios, renames y paths con caracteres especiales: escanear de manera segura o declarar su exclusión. No seguir enlaces fuera del snapshot. Aislar el entorno y no ejecutar nada del repo durante el análisis.

3. Para delta, comparar con una base identificada; no asumir que `HEAD` equivale a la base de PR. Si cambia una regla, política, archivo de configuración, versión de escáner, soporte de lenguaje o base de avisos, invalidar partes afectadas y reanalizar. Los lockfiles se revisan completos al cambiar y cuando cambia la base de avisos; una CVE puede aparecer aunque no haya diff.

4. La huella de caché debe incluir contenido/snapshot, contexto que el escáner lee, versión/hash de binario, reglas, flags, plataforma, versión/esquema de normalización y fecha/hash de la base. Si no se conocen todas las entradas, no reutilizar. Registro de resultados no incluye el valor literal del secreto.

5. Evitar TOCTOU: antes de aplicar una decisión verificar de nuevo los hashes; para enforcement remoto, CI ejecuta la revisión sobre los mismos commits/objetos del servidor. Los hallazgos sin huella verificable o de cobertura parcial se etiquetan.

Estados propuestos: `PASS` solo si etapas requeridas terminaron y no hay hallazgos por encima de la política; `FINDINGS` si hay hallazgos bloqueantes; `INCOMPLETE` si falta un escáner/base, falla el aislamiento exigido, se agotó tiempo, el esquema es desconocido o hubo alcance no soportado. Un aviso de antigüedad solo bloquea si la política lo establece; si no, el reporte siempre muestra edad. En modo informativo los errores siguen visibles. La ausencia de hallazgos jamás se formula como «seguro».

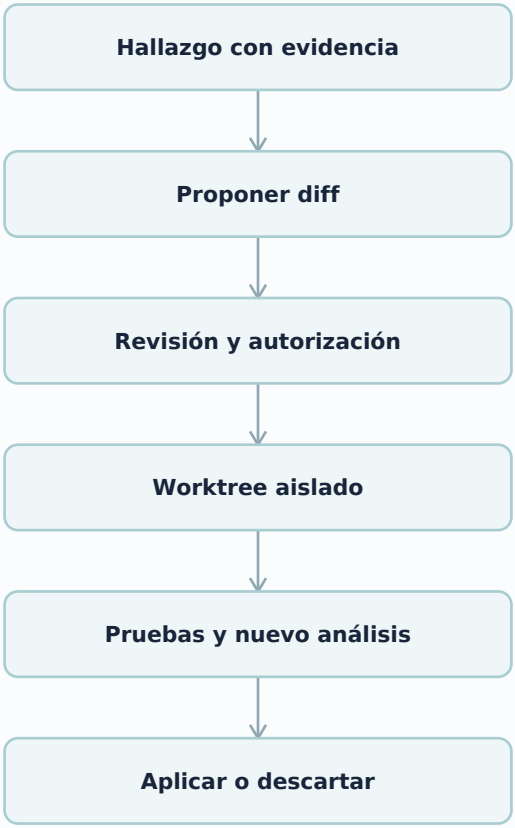
9.4 Flujo de análisis



- **Instalación propuesta:** usuario obtiene versión fijada de OpenQodex y herramientas por canal verificable; inspecciona binarios, hashes y licencias; ejecuta una configuración guiada. Nunca usar `npx` sin fijar versión en un entorno de alta confianza; los paquetes y scripts de instalación tienen su propia superficie de ataque.
- **Primer análisis:** detectar lenguaje/lockfiles sin ejecución del repositorio, informar qué se encontró, preparar base OSV de cada ecosistema con autorización de red y guardar fecha/hash; posteriormente analizar en modo desconectado. En un entorno completamente aislado, transferir la base previamente descargada con procedencia controlada.
- **Análisis incremental:** por cambios de archivos, configuración y reglas; volver a revisar lockfiles completos cuando corresponda. Si el motor necesita más contexto que el diff, entregarle el snapshot y filtrar con una política explícita, sin eliminar detecciones vinculadas a cambios de dependencias o configuración.

- **Detección y validación:** reproducir regla o match con IDs y rutas; distinguir coincidencia de demostración de explotación. Desduplicar alias y resultados solo cuando identidad y ubicación lo justifiquen. El LLM, si se añade, sugiere y cita líneas; nunca oculta resultados sin evidencia y revisión.
- **Reportes:** orden por severidad, cambio, exposición conocida y confianza; mostrar detecciones y cobertura desconocida. Un «riesgo desconocido» no recibe cero por defecto.
- **Monitoreo:** opt-in. Si el equipo exige alerta por una nueva CVE sin cambio de Git, debe ejecutar un calendario o actualización de base y reanalizar; en offline permanente la herramienta solo conoce la última base disponible.

9.5 Remediación y rollback: fase posterior



Un parche es una propuesta, no un resultado seguro. Primero se genera diff y se muestra alcance, archivos y efectos esperados; una persona autoriza el parche y por separado la ejecución de tests o gestores de paquetes del repo. Usar snapshot/worktree aislado con permisos mínimos, sin red ni secretos cuando sea posible; tests pueden ejecutar código arbitrario y deben tener sandbox apropiado. Verificar que desaparezca el hallazgo original, que no aparezcan nuevos riesgos, y ejecutar regresiones pertinentes. Aplicar el diff solo al mismo contenido aprobado; si cambió, solicitar nueva revisión. Rollback mediante `git diff`/commit reversible o restauración del snapshot, nunca con limpieza destructiva automática del working tree. No hacer merge ni deploy automáticamente en MVP.

9.6 Agentes especializados: decisión condicionada



El planificador no necesita LLM. Agentes de código, dependencias, infraestructura e investigación serían roles lógicos solo si un benchmark muestra beneficio; no tienen que ser procesos autónomos. Un único revisor opt-in con contexto mínimo es el primer experimento de IA. Un revisor independiente puede compartir sesgos o datos contaminados; dos agentes de acuerdo no demuestran verdad. Si se adoptan múltiples roles: mensajes tipados con ID y procedencia, trabajos idempotentes, límite de profundidad/concurrencia/tokens, timeout y reintentos con backoff, permisos por rol, contextos separados y sin secretos compartidos. Mantener la última decisión sensible en el usuario.

9.7 Seguridad de la herramienta

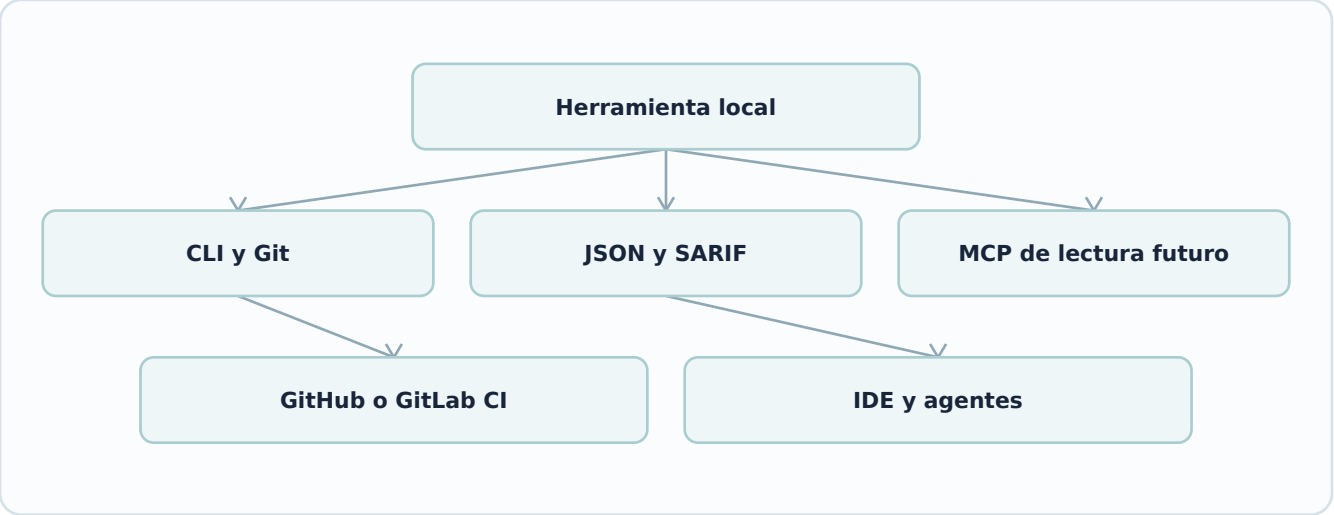
- **Cadena propia:** fijar dependencias y binarios por versión y hash; verificar firmas/procedencia con raíz de confianza independiente cuando estén disponibles; mantener inventario SBOM y avisos de dependencias; firmar releases y documentar cómo verificar.
- **Instalación y configuración:** el proyecto analizado se considera no confiable, incluso sus archivos de configuración de seguridad; una lista de comandos o plugin se aprueba fuera del repo para el hash exacto. No ejecutar scripts de instalación o del repositorio al inspeccionarlo. El análisis de un archivo puede explotar un parser: limitar proceso, tamaño y tiempo; actualizar escáneres.
- **Aislamiento:** usuario sin privilegios, filesystem mínimo de solo lectura y escritura solo a un directorio temporal, no montar llaves, token cloud, socket Docker ni carpeta personal; red denegada cuando el modo lo promete. Separar subprocesos, recursos y secretos de OSV/revisor. Documentar compatibilidad real por SO.
- **Secretos y datos privados:** almacenar ubicación y huella de secreto sintético/redactado, jamás la cadena completa en JSON, logs, prompt o mensajes. Filtro determinista antes de cualquier IA no garantiza eliminación total de secretos. Cifrado del disco y permisos del usuario dependen del sistema; no prometer cifrado propio sin implementarlo.
- **MCP:** si se agrega, únicamente lectura, transporte local restringido y acceso de solo lectura a un conjunto fijado; autenticar cuando se publique HTTP, scopes por workspace, validar esquema/tamaño,

evitar tool descriptions no confiables como autoridad. El SDK no aporta seguridad por sí solo [S02, S62].

- **Auditoría y errores:** registrar ID del análisis, actor local, snapshot, versiones, política, tiempos y excepciones sin contenido sensible. Separar error de hallazgo; límite de logs; datos temporales limpiados de manera segura. Proteger contra path traversal, symlinks, TOCTOU, salida malformada y saturación.
- **Privacidad:** local por defecto sin telemetría; en línea solo con consentimiento explícito para actualizar bases o usar API; revelar qué sale (p. ej., nombres/versiones de paquetes o fragmentos) y permitir revocación.

10. Integraciones, notificaciones y automatización

10.1 Prioridad y límites



Integración	Mecanismo recomendado	Prioridad / límites
Repositorios locales / Git	CLI + `pre-commit` o `pre-push` opt-in	P0. Los hooks pueden omitirse; no son frontera de seguridad.
GitHub/GitLab	Comando en pipeline, artifact JSON/SARIF, política del servidor	P1. Fijar acciones, aislamiento y permisos. Compatibilidad de ingestión SARIF y licencias dependen de plataforma/plan.
Codex / Claude Code / Cursor / Cline / OpenCode	Invocar CLI y consumir reporte; hooks solo según interfaz documentada del host	P1. No asumir que se interceptan acciones ejecutadas en otras rutas.
VS Code y Copilot	CLI desde terminal/tarea; después extensión o SARIF	P2. Una extensión tiene costo de mantenimiento propio.
Gemini CLI / Aider	CLI o hook Git; adaptar solo tras pruebas reales	P2. Permisos e interfaz cambian según versión.
Servidor MCP	Lectura de reportes seleccionados y datos redactados	P2. No invocar shell ni aplicar correcciones en respuesta al modelo; autenticación si es remoto.
Docker / servicios	Empaquetado y análisis solo en contextos confiables y autorizados	P2. No montar Docker socket en un escáner de repos no confiables.
Frameworks/API	Contrato JSON versionado y proceso local	P2. Diseñar SDK solo si los integradores lo necesitan.

10.2 Notificaciones

Canal	Integración y costo	Riesgos / decisión
Local	Resumen de terminal y archivo	P0, sin proveedor. No emitir detalles de secretos en pantalla compartida.
Webhook	HTTPS con HMAC, timestamp, nonce y reintentos idempotentes	P2; URL y secreto protegidos; salida solo a destino permitido.
Discord	Webhook con rate limits y secretos	P3; no poner código o credenciales en mensajes; precios/condiciones de la plataforma pueden cambiar [S69].
Telegram	Bot API; límites gratuitos y opción de broadcast pagado según FAQ	P3; verificar chat ID, proteger token y considerar privacidad; no usar mensajes como canal de ejecución [S49].
Correo	SMTP o servicio; proveedor puede cobrar por volumen	P3; costo no estimado sin proveedor ni volumen, riesgo de filtración y suplantación.
WhatsApp	API empresarial/proveedor; precio y elegibilidad dependen de país, categoría y plantilla	Fuera del MVP; consultar tarifa vigente al contratar, no inventar una cifra.
OpenClaw	Gateway multi-canal con permisos y plugins [S74]	Descartado para notificaciones simples por superficie y mantenimiento añadidos.

Primera versión: solo informes, no aprobación ejecutable por chat. Si en una fase posterior se autorizan acciones, el mensaje deberá abrir una sesión segura local o web autenticada, con identidad y rol, desafío de un solo uso, expiración, `hash` exacto del cambio/acción, protección anti-replay y auditoría. La aprobación será para un recurso y operación concretos; nunca aceptar «sí» o comandos de cualquier chat como autorización suficiente. Verificar identidad y permisos en el punto de ejecución; caducar si cambió el contenido.

10.3 Qué puede automatizarse

Etapa	Automatización	Supervisión necesaria
Identificación de stack, análisis, agregación y reporte	Automática tras configuración; marcar entradas omitidas	Revisar cobertura/errores y política.
Actualizar base de advisories	Programable con red autorizada y verificación	Aceptar fuente y vigencia; offline impide conocer avisos nuevos.
Priorización	Automática con motivos visibles	Revisar supresiones y señales inciertas; no ocultar FNs.
Sugerencia de parche	Futura, opt-in	Ver diff, licencia, privacidad y resultado.
Ejecutar tests de proyecto, instalar o actualizar paquetes	No automática sobre repo no confiable	Autorización y aislamiento: scripts de paquete pueden ejecutar código.
Aplicar, enviar, hacer merge o desplegar	Fuera de MVP	Autorización explícita y controles de repositorio.

11. Costos, infraestructura y tokens

11.1 Precios verificables y fórmula

Al 28/10/2026, la tabla estándar consultada muestra Gemini 2.5 Flash-Lite a USD 0.10 por millón de tokens de entrada de texto y USD 0.40 por millón de tokens de salida, incluidos tokens de razonamiento facturables. OpenAI muestra GPT-6.1-sol estándar, contexto corto, a USD 2.00 por millón de tokens de entrada y USD 10.00 por millón de salida [S46–S47]. Son ejemplos, no recomendaciones de modelo ni una garantía de que estén disponibles para cada cuenta. No se aplican aquí descuentos, caché, red, impuestos, herramientas externas o almacenamiento; precios y condiciones pueden cambiar.

Fórmula: costo de API = `(tokens_entrada × precio_entrada + tokens_salida × precio_salida) / 1,000,000`. Un modelo local no tiene precio API por token, pero incurre en hardware, energía y tiempo. Los tokens son supuestos de simulación, no consumo medido.

Pequeño	
Modelo A sin LLM	0 tokens; USD 0 de API
Modelo B: Flash-Lite, tokens entrada/salida	10,000 / 2,000
Costo B USD/análisis	0.0018
Modelo C: 10 veces tokens de B	100,000 / 20,000
Costo C USD/análisis	0.018

Mediano	
Modelo A sin LLM	0 tokens; USD 0 de API
Modelo B: Flash-Lite, tokens entrada/salida	40,000 / 8,000
Costo B USD/análisis	0.0072
Modelo C: 10 veces tokens de B	400,000 / 80,000
Costo C USD/análisis	0.072

Grande	
Modelo A sin LLM	0 tokens; USD 0 de API
Modelo B: Flash-Lite, tokens entrada/salida	200,000 / 40,000
Costo B USD/análisis	0.0360
Modelo C: 10 veces tokens de B	2,000,000 / 400,000
Costo C USD/análisis	0.360

Usar GPT-6.1-sol estándar con las mismas cantidades B daría USD 0.04, 0.16 y 0.80, respectivamente; con C, USD 0.40, 1.60 y 8.00. Las cifras son aritmética de ejemplo, no equivalencia de calidad ni comparación de capacidades. El escenario grande puede requerir dividir contexto; si existen recargos de contexto largo, tool calls o precios distintos, recalcular. Enrutamiento o caché reducen costo solo si se

miden en el uso real. Las cuotas de productos de consumo no se pueden convertir a tokens API sin sus términos.

11.2 Comparación técnica: escenarios hipotéticos

Cuándo usar	
A: determinista	Todas las revisiones ordinarias
B: híbrido bajo demanda	Casos priorizados con evidencia y consentimiento
C: multiagente	Solo si un experimento controlado supera A/B.
Infraestructura	
A: determinista	CLI y binarios, CPU, disco/base
B: híbrido bajo demanda	A + modelo local o API
C: multiagente	B + orquestación, estado, control de permisos y herramientas.
CPU/GPU/memoria	
A: determinista	CPU; sin GPU obligatoria; consumo del escáner y repo
B: híbrido bajo demanda	GPU opcional; modelo local requiere memoria según pesos y contexto
C: multiagente	Más procesos/contextos; la concurrencia aumenta RAM y costo.
Tiempo	
A: determinista	Desde segundos a minutos como hipótesis, no medido
B: híbrido bajo demanda	A + inferencia y validación
C: multiagente	A + varias inferencias, coordinación y verificación.
Calidad esperada	
A: determinista	Reproducible para mismos datos; cobertura limitada
B: híbrido bajo demanda	Puede añadir contexto o ruido; debe medirse
C: multiagente	Puede ampliar análisis, sin garantía de mejora; mayor superficie de ataque.
Mantenimiento	
A: determinista	Versiones de escáner, reglas y fuentes
B: híbrido bajo demanda	Además prompts, modelos, privacidad y validación
C: multiagente	Además coordinación, permisos, observabilidad y fallas distribuidas.
Escala	
A: determinista	Paralelismo limitado por CPU/I/O

B: híbrido bajo demanda	Añade límites de tokens y modelo
C: multiagente	Limitar concurrencia, presupuesto y cascada de reintentos.

Para planear, probar en laptop de 4 núcleos/8 GB y en runner de 8 núcleos/16 GB, con repos pequeños, medianos y grandes. Son **configuraciones propuestas**, no requisitos probados. Registrar p50/p95, CPU, pico RSS y almacenamiento real por escáner; no asumir que un modelo 4B o una base concreta cabe en cierta memoria. Una aproximación de peso de 4B a 4 bits es 2 GB solo para parámetros ideales, sin metadatos, caché KV, runtime, RAM del sistema ni garantía de calidad.

11.3 Presupuesto operativo mensual ilustrativo

Individual local	
Uso hipotético	100 cambios pequeños/mes
API A	USD 0
API B Flash-Lite	USD 0.18 si todos usan IA
Obligatorios / opcionales y límites	Equipo, energía y base actualizada son costos reales; IA opcional.

Equipo pequeño	
Uso hipotético	500 cambios pequeños/mes
API A	USD 0
API B Flash-Lite	USD 0.90
Obligatorios / opcionales y límites	Trabajo humano, gestión de versiones y CI dependen de entorno.

Proyecto mediano	
Uso hipotético	200 cambios medianos/mes
API A	USD 0
API B Flash-Lite	USD 1.44
Obligatorios / opcionales y límites	Incluye solo tokens simulados; no hosting, impuestos ni datos enviados.

Intensivo	
Uso hipotético	1,000 cambios grandes/mes
API A	USD 0
API B Flash-Lite	USD 36.00
Obligatorios / opcionales y límites	El escenario C de 10 veces tokens sería USD 360; puede exceder límites y contexto.

GitHub documenta minutos incluidos en cada plan y uso gratuito de runners estándar para repositorios públicos; las condiciones de privados, almacenamiento y límites deben revisarse para la cuenta y periodo [S48]. Un plan gratuito no significa capacidad infinita. No se asume precio por almacenamiento, WhatsApp, dominios, servidores, tests de seguridad externos o mantenimiento; calcularlos con el proveedor elegido. Para energía, aplicar `kW incrementales × horas × tarifa local`: por ejemplo, a tarifa

hipotética de USD 0.15/kWh, 80 W incrementales durante 10 minutos cuestan USD 0.002. No es tarifa eléctrica mexicana ni medición del producto.

11.4 Presupuesto de desarrollo y mantenimiento

Descubrimiento y spike de integración, incluidas primeras cuatro semanas	
Horas estimadas del equipo	120–160 h
Referencia a USD 25–50/h	USD 3,000–8,000
Estado	Estimación, depende de acceso y APIs upstream.

MVP completo adicional	
Horas estimadas del equipo	200–320 h
Referencia a USD 25–50/h	USD 5,000–16,000
Estado	Estimación; no incluye despliegue gestionado o auditoría externa.

Total hasta MVP	
Horas estimadas del equipo	320–480 h
Referencia a USD 25–50/h	USD 8,000–24,000
Estado	Adicionales de seguridad, compatibilidad Windows y cambios de upstream pueden incrementarlo.

Mantenimiento luego del MVP	
Horas estimadas del equipo	20–60 h/mes
Referencia a USD 25–50/h	USD 500–3,000/mes
Estado	Escenario, no obligación; depende de adopción, escáneres y seguridad.

Las tarifas son hipótesis simples de carga para poder calcular, no datos verificados de mercado. Excluir contratación, IVA y prestaciones si no se han estimado explícitamente. Una auditoría de seguridad independiente o pruebas remuneradas requieren cotización; no hay precio confiable universal aquí. Evaluar primero el uso sin IA y con una revisión humana; la IA solo se justifica si **mejora utilidad neta** respecto a A considerando tiempo humano, falsas negativas, privacidad y costo, no por ser barata en tokens.

11.5 Reducir consumo sin degradar silenciosamente cobertura

Caché sensible a todos los insumos, diffs con invalidación, ejecución selectiva explicable, lotes y desduplicación. Usar LLM solo para hallazgos ambiguos de mayor prioridad, extraer fragmentos mínimos y contexto relevante por referencia, no mandar el repositorio entero. Aplicar presupuesto por análisis, límite de intentos, truncado con estado incompleto y modo de abstención. Reutilizar respuesta solo para la misma evidencia y versión de modelo/prompt. Ninguna técnica debe reducir el análisis de dependencias cuando se actualiza la base sin cambios de lockfile.

12. MVP, requisitos y aceptación

12.1 Alcance

Imprescindible	Posterior	Fuera de alcance inicial
Evaluar y fijar base; CLI local; Python/JS como stacks piloto; detectar lockfiles soportados; escaneo de secretos y SCA OSV offline; análisis de código en reglas de licencia verificada; JSON/SARIF y salida humana; snapshot y cobertura; modo incompleto explícito; pruebas de seguridad; guía de instalación.	MCP de solo lectura; más lenguajes; integración GitHub/GitLab; revisión contextual opt-in; programador de actualizaciones; notificaciones informativas; remediación guiada.	Control universal de agentes, monitor kernel/eBPF, aplicación automática de parches, bot de WhatsApp con comandos, servidor multiusuario, DAST de servicios ajenos, interfaz SaaS, multiagente obligatorio.

Hipótesis principal: usuarios que desarrollan con IA y código privado prefieren una revisión local que indique exactamente qué se analizó y cuáles datos estaban vigentes, aun cuando detecte menos clases que un producto comercial. Esta hipótesis requiere validación.

12.2 Historias y criterios verificables

Historia	Aceptación
Como desarrollador, quiero preparar el análisis con pocos pasos	En Linux o macOS soportado, una guía desde instalación limpia identifica requisitos, obtiene herramientas fijadas, solicita permiso de red y ejecuta una muestra; registrar tiempo y errores en 5 pilotos.
Quiero analizar sin enviar código o dependencias	Con bases y binarios instalados, el modo estricto pasa una prueba de bloqueo de red del entorno; si no hay aislamiento verificable, devuelve `INCOMPLETE`; no activar revisor externo.
Quiero saber qué cambios se revisaron	JSON y salida humana incluyen snapshot, base, archivos analizados, omisiones y hashes; análisis de push maneja varias refs o falla explícitamente.
Quiero conocer vulnerabilidades de dependencias	Lockfile soportado con fixture de versión vulnerable produce aviso correcto; misma versión corregida deja de coincidir; base ausente o demasiado antigua se muestra, no se interpreta como limpia.
Quiero revisar secretos y código sin exponerlos	Hallazgo con secreto de prueba muestra ruta, línea y evidencia redactada; los logs no contienen el valor. Las reglas SAST y sus licencias están versionadas.
Quiero decidir sobre resultados incompletos	Falta de herramienta, timeout, output inválido, symlink rechazado o interrupción produce un estado visible; política de CI puede bloquear por `INCOMPLETE`.
Quiero evitar cambios inesperados	El modo de análisis no escribe en el repo ni instala hooks sin consentimiento; pruebas adversarias lo comprueban.
Quiero hacer excepciones responsables	Supresión guarda responsable local, motivo, alcance y expiración; mostrarla en reporte y no aceptar una configuración externa como autorización.
Quiero comparar con lo existente	El piloto registra tiempo de instalación, hallazgos confirmados y carga humana contra OpenQodex y escáneres separados en el mismo corpus.

13. Plan de validación y pruebas de concepto

13.1 Evidencia disponible y conflictos

Pearce y colegas evaluaron escenarios concretos de Copilot en 2021/2022 y reportaron una proporción significativa de ejemplos vulnerables; Perry y colegas estudiaron usuarios en tareas de seguridad; Sandoval y colegas encontraron resultados matizados en tareas C. La comparación no demuestra que todo código de IA sea inseguro ni que el riesgo tenga un porcentaje actual fijo [S04–S06]. OWASP Benchmark proporciona casos con etiquetas para herramientas SAST/DAST, con diferencias entre lenguajes y escenarios. SastBench se presenta como benchmark de triaje; validar licencia, splits y madurez antes de adoptarlo [S40–S41].

Las cifras de OpenQodex son declaradas por su propio autor y difieren según benchmark; no se han reproducido. Los comentarios públicos y issues prueban que existen experiencias de ruido o limitaciones, pero no su prevalencia [S08, S56–S57]. En una discusión de Hacker News sobre un revisor de código, participantes discreparon sobre el costo de los falsos positivos; un proveedor afirmó haber corregido un problema en la versión evaluada. Es una observación cualitativa y no un benchmark independiente ni representativo [S75]. Ninguna fuente consultada demuestra que nuestro producto sea mejor ni que los usuarios lo necesiten.

13.2 Prueba de concepto ejecutada aquí

Objetivo: verificar una selección de cambios y consolidación paralela en un repositorio propio efímero. **Hipótesis:** una selección de archivos modificados permite conservar candidatos introducidos y excluir un candidato anterior en un caso simple. **Herramientas:** Git, Python estándar, tres detectores didácticos: AST para `eval`, un patrón de secreto sintético y un match exacto de una versión con datos transcritos de OSV. No son Semgrep, Gitleaks ni OSV-Scanner. No se ejecuta el código del fixture ni se instalan paquetes.

Procedimiento: crear tres archivos y commit de base; modificar `app.py` y `package-lock.json`; usar `git diff --name-only -z HEAD`; ejecutar detectores en paralelo sobre todos los archivos y después sobre los modificados. Fuente OSV consultada: GHSA-35jh-r3h4-6jhm / CVE-2021-23337, lodash anterior a 4.17.21 [S50]. Un `package-lock.json` con `4.17.20` solo produce una coincidencia con el ejemplo; no indica alcanzabilidad ni explotación real.

Resultado observado	Completo	Incremental
Archivos	`app.py`, `old.py`, `package-lock.json`	`app.py`, `package-lock.json`
Candidatos	4: `eval` nuevo, `eval` previo, clave sintética y match de dependencia	3: `eval` nuevo, clave sintética y match de dependencia
Tiempo de una ejecución	7.39 ms	3.31 ms

Limitaciones: tiempos de un único ensayo, incluyen scheduler y caché, son irrelevantes para estimar rendimiento real; detectores triviales no tienen evaluación de precisión; no se consultó la API OSV desde el script; el aviso se transcribió manualmente. No cubre untracked, staged por separado, renames, múltiples refs, cambios de configuración ni TOCTOU. El apéndice contiene el programa reproducible. **Conclusión:** la composición y selección básica funcionan en el ejemplo; no valida el producto. **Siguiente:** repetir con herramientas oficiales, base OSV offline y repositorios anotados.

13.3 Experimentos reproducibles pendientes

1. **Base candidata:** en entorno con red autorizada, fijar versión/commit de OpenQodex, revisar licencia y hash, instalar en una VM efímera y ejecutar su demo y modo de escaneo. Registrar qué archivos, binarios, conexiones y mensajes genera; comparar la versión y flags de su documentación. Aún no ejecutado.

2. **OSV offline real:** instalar versión fijada de OSV-Scanner; preparar base de ecosistema con el método oficial y guardar fecha/hash; desconectar la red de la VM; ejecutar `osv-scanner --offline ./fixture` según versión; registrar salida y error esperado sin base. La sintaxis y uso de la base están documentados por OSV, pero no se han probado aquí [S51].

3. **Composición real:** integrar Gitleaks y OSV con el orquestador; introducir fixture sintético y lockfile versionado; comparar salida normalizada, fallo por timeout y versión fija. Aún no ejecutado.

4. **Incremental y efectos globales:** modificar un archivo, cambiar solo una regla, actualizar solo la base OSV y probar renames, submódulos, múltiples refs, untracked, staged y symlinks. Comprobar que invalida caché y no pierde avisos de dependencias.

5. **IA frente a determinista:** mismo conjunto de hallazgos y evidencia para A, B y C; modelo ciego a etiqueta; límites y costos medidos; adjudicación humana. Un modelo que reduce reportes mediante supresión puede aumentar falsos negativos.

6. **Notificación simulada:** emisor local escribe un evento firmado en archivo y receptor en loopback verifica autenticidad, timestamp y nonce. No enviar nada a servicios externos. Después probar replay, mensajes duplicados y expiración.

7. **Amenazas del analizador:** repositorio sintético con symlink hacia fuera, nombres extraños, archivos enormes, config con comando no aprobado, parser output inválido, secretos de prueba y texto de prompt injection; comprobar no ejecución y no fuga.

13.4 Métricas y diseño científico

Métrica	Definición y modo de medición
Precisión	`TP / (TP + FP)` en hallazgos adjudicados; reportar intervalos e «indeterminado».
Recall	`TP / (TP + FN)` solo donde se conozca un conjunto de vulnerabilidades etiquetadas; no se puede inferir de un escaneo de producción sin ground truth.
FPR / FNR	`FP/(FP+TN)` y `FN/(FN+TP)`; declarar unidad de conteo, universo y categorías.
Riesgos críticos	Cantidad de TP críticos ponderados por etiqueta; no atribuir «critical» solo por parecer grave.
Tiempo y recursos	p50/p95 de tiempo de pared, CPU, pico RSS y disco, con hardware y versiones; separar descarga fría/caché.
Tokens y costo	Medir entrada/salida facturada y costo según factura/precio vigente, por resultado aceptado.
Calidad de parche	Proporción aplicada que pasa test de seguridad relevante, regresión y revisión humana; reportar nuevas fallas y parches no probables.
Productividad y adopción	Tiempo hasta primer reporte útil, minutos de triaje, interrupciones y uso sostenido; evitar medir productividad solo en líneas de código.

Protocolo: corpus mixto con OWASP Benchmark, SastBench tras verificar licencia y repositorios voluntarios; sembrar casos reproducibles y controlar contaminación de benchmark en modelos. Separar desarrollo y evaluación por repositorio y tiempo; 2 revisores independientes para muestras, resolver

desacuerdos, registrar indeterminados y cambios de etiquetas; muestrear también negativos para buscar falsos negativos. Congelar contenedores, versiones, reglas, bases y fecha; ejecutar al menos 3 repeticiones de rendimiento en frío/caliente. Comparar escáneres individuales, OpenQodex sin cambios, Guardia A, y B opcional bajo el mismo presupuesto. Presentar hallazgos apareados, IC y tamaño de muestra, no solo porcentajes.

Umbrales de salida propuestos, no resultados: 5/5 instalaciones piloto completadas con guía; ninguna salida de red en modo estricto bajo prueba controlada; 100 % de fallas inyectadas notificadas como incompletas; cero secretos sintéticos en logs/reportes; al menos 70 % de participantes que encuentren útil el informe y 3 de 5 que repitan en una segunda sesión. Para seguridad, cero fallos observados no prueba ausencia de fallos. Antes de afirmar reducción de ruido o recall superior exigir corpus suficientemente etiquetado y comparación estadística.

14. Licencias, comunidad y sostenibilidad

14.1 Comparación

Licencia	Distribución/uso y obligaciones generales	Encaje
MIT	Permisiva; conservar copyright y licencia; sin concesión explícita de patentes como Apache-2.0	Muy simple; compatible con muchas integraciones.
Apache-2.0	Permisiva, licencia explícita de patentes y condiciones/notices para redistribución	Recomendación para contribución compatible con OpenQodex, sujeta a revisión de terceros.
GPL	Copyleft al distribuir obras derivadas/combinadas según condiciones	Favorece reciprocidad; analizar compatibilidad y distribución.
AGPL	Copyleft con obligaciones adicionales ante interacción por red de versiones modificadas	Renovate usa AGPL-3.0-only; valorar integración por proceso y obligaciones concretas [S20].
LGPL	Copyleft para biblioteca/componente con condiciones para aplicaciones combinadas	Semgrep CE se distribuye con LGPL-2.1-or-later; revisar redistribución y modificaciones [S11].
Source-available	Código visible con restricciones que pueden no cumplir OSI	No llamarlo open source. Pipelock separa core Apache y enterprise bajo otra licencia; Semgrep Rules tiene licencia separada [S12, S44].

Las licencias OSI permiten ciertas libertades y no se reducen a que haya un repositorio público [S52]. Apache-2.0 propio no cambia la licencia de reglas, motores, modelos, bases, logos o contenido de terceros. Para cada release generar SBOM y `THIRD_PARTY_NOTICES`, reproducir avisos exigidos, verificar compatibilidad de los artefactos concretos y evitar incorporar reglas con restricciones no aceptadas. Proceso separado puede reducir acoplamiento técnico, pero no garantiza automáticamente una conclusión legal; consultar asesoría al distribuir comercialmente.

14.2 Gobierno, seguridad y comunidad

Repositorio propuesto si se justifica uno separado: `docs/` con amenaza y guías; `src/` con CLI/adapters/policy; `rules/` solo reglas propias o compatibles; `fixtures/` sin secretos reales; `tests/` unitarias, contractuales y aislamiento; `schemas/` para JSON; `examples/` con CI opt-in; `.github/` con plantillas y workflows. Añadir `README`, `LICENSE`, `NOTICE`, `CONTRIBUTING`, `CODE_OF_CONDUCT`, `SECURITY` con reporte privado, `CHANGELOG` y soporte de versiones.

Publicar roadmap y decisiones técnicas, usar revisiones independientes para cambios de seguridad, versionado semántico de schemas, firmas/procedencia de releases, pruebas de dependencias y política de actualización. Evitar aceptar parsers, reglas o plugins sin pruebas y revisión de licencia. Definir responsables de triage, matriz de plataformas soportadas y tiempo de respuesta como objetivo medido, no promesa prematura. Posibles ingresos futuros: soporte, integraciones, operación gestionada o formación; el núcleo no debe depender de exfiltrar código ni crear restricciones incompatibles con la licencia. No hay validación de disposición a pagar.

15. Roadmap y backlog

15.1 Plan de fases

La duración asume 1–3 personas, disponible el hardware y sin esperar aprobaciones externas. Con dos personas al 50 % del tiempo son 40 h de equipo por semana. Las fases son parcialmente solapables; el total es una banda de planeación, no una promesa de entrega.

0. Validación	
Objetivo y tareas	10–15 entrevistas semiestructuradas; mapa de flujo actual; 5 pilotos reclutados; comparar competidores
Dependencias / prioridad	Sin código; P0
Esfuerzo estimado	30–50 h
Riesgo y aceptación / entregable	Sesgo de muestra; entregar entrevistas anonimizadas y criterios de continuar/parar.

1. Selección de bases	
Objetivo y tareas	Ejecutar OpenQodex, SecGate y OSV offline; auditar licencias, red, extensiones y límites
Dependencias / prioridad	Acceso a repositorios/paquetes; P0
Esfuerzo estimado	40–60 h
Riesgo y aceptación / entregable	La API o licencia no permita integración; informe reproducible y decisión upstream/adaptador.

2. Prototipo	
Objetivo y tareas	Añadir a un entorno aislado SCA offline, manifiesto de cobertura y snapshot mínimo
Dependencias / prioridad	Fase 1; P0
Esfuerzo estimado	70–100 h
Riesgo y aceptación / entregable	Integración inestable; reporte JSON para fixture y prueba sin red.

3. MVP	
Objetivo y tareas	Harden runner, errores, caché, documentación, análisis de secretos y reglas, pruebas en 5 proyectos autorizados
Dependencias / prioridad	Fase 2; P0
Esfuerzo estimado	160–240 h
Riesgo y aceptación / entregable	Falsos negativos, plataformas y privacidad; cumplir §12.2 y §13.4.

4. Integraciones	
Objetivo y tareas	CI GitHub/GitLab y adaptación a agentes, sin MCP de escritura

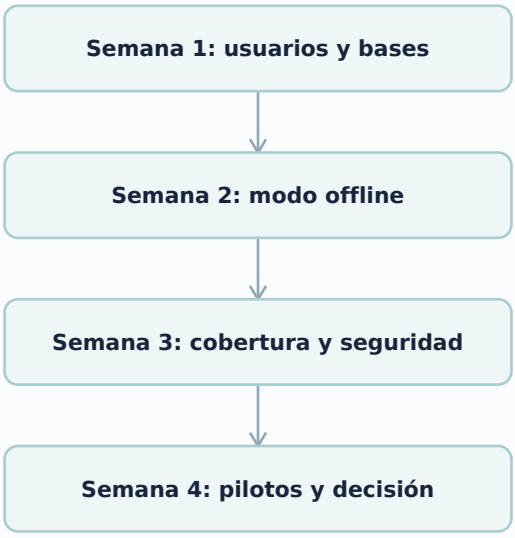
Dependencias / prioridad	MVP y consentimiento; P1
Esfuerzo estimado	60–100 h
Riesgo y aceptación / entregable	Eludir hooks, planes/formatos; CI verifica commit y muestra incompletos.

5. Automatización inteligente	
Objetivo y tareas	Revisor opcional solo con datos permitidos; validar vs baseline
Dependencias / prioridad	Corpus etiquetado y privacidad; P2
Esfuerzo estimado	100–180 h
Riesgo y aceptación / entregable	Costos y FNs; mantenerlo fuera por defecto salvo resultados.

6. Comunidad	
Objetivo y tareas	Documentación, guía de contribución, disclosure privado, release firmada
Dependencias / prioridad	MVP y seguridad; P1, parcialmente paralelo
Esfuerzo estimado	40–80 h
Riesgo y aceptación / entregable	Capacidad de mantenimiento; una persona externa instala desde release y reproduce demo.

Horas de fases pueden solaparse y no son sumables con exactitud al total de §11, que agrupa el trabajo hasta MVP. Fases 4–6 son estimaciones posteriores al MVP. Gastos en herramientas son opcionales salvo equipo, energía y tiempo; presupuestar una auditoría independiente por cotización y no suponer que ya está incluida.

15.2 Primeras cuatro semanas



Semana	Entregables y decisiones	Trabajo paralelo posible
1	5–8 entrevistas iniciales; inventario de licencias; ejecutar OpenQodex/SecGate y OSV offline; registrar conexiones; decidir si hay punto de extensión.	Una persona investiga usuarios mientras otra realiza el spike; no se requiere fork público.
2	Fixture JS/Python, base OSV local con fecha/hash, integración mínima y reporte inicial; reproducir sin red.	Especificación JSON y diseño de pruebas de seguridad.
3	Snapshot Git, invalidación básica, estados de fallo, redacción de secretos y pruebas de configuración no confiable.	Preparar guías de usuario y metodología del piloto.
4	5 sesiones de instalación/revisión; comparar OpenQodex sin cambios; medir fricción y hallazgos, decisión explícita de contribuir, adaptar o detener.	Documentación de hallazgos y conversación con upstream, solo con autorización antes de enviar mensajes.

15.3 Backlog inicial listo para trasladar

ID y título	Descripción / criterio de cierre	Prioridad / dependencia
G-01. Definir corpus y contrato de análisis	Crear fixtures sintéticos con etiquetas, formatos y estado incompleto; revisión por otra persona.	P0 / ninguna
G-02. Reproducir OpenQodex	Fijar versión, demo, analizar modo solo escáner y registrar conexiones/limitaciones con comandos reproducibles.	P0 / ninguna
G-03. Verificar licencias	Registrar SPDX, avisos, reglas y redistribución de todos los artefactos propuestos; resolver dudas antes de publicar.	P0 / ninguna
G-04. Probar OSV offline	Preparar base, desconectar red, encontrar versión de prueba y fallar claramente con base ausente.	P0 / ninguna
G-05. Diseñar extensión o adaptador	Demostrar punto de extensión de OpenQodex y fallback a proceso JSON si no existe.	P0 / G-02, G-04
G-06. Resolver snapshot	Prototipo para commit/worktree; registrar hash, alcance y omisiones; pruebas de rutas/symlink.	P0 / G-01
G-07. Runner con límites	argv sin shell, variables mínimas, timeout, tamaño de salida y prueba de aislamiento aplicable al SO.	P0 / G-05
G-08. Manifiesto de cobertura	Versionar esquema: insumos, base, edad, herramientas, estados y hashes; documentar `PASS/FINDINGS/INCOMPLETE`.	P0 / G-05, G-06
G-09. Redactar secretos	Ruta/línea/hash de evidencia, sin valor en logs ni notificaciones; tests con valores sintéticos.	P0 / G-07
G-10. Reporte legible y SARIF	Mostrar los hallazgos y motivos, cobertura incompleta y antigüedad; salida JSON/SARIF válida.	P0 / G-08, G-09
G-11. Caché e invalidación	Incluir contenido, config, versiones, reglas y base; tests de cambios en inputs y rollback.	P1 / G-08
G-12. Pruebas de push y CI	Varias refs, refs inexistentes, push forzado, omisión de hooks y verificación del servidor.	P1 / G-06
G-13. Laboratorio adversarial	Config maliciosa, path traversal, symlink, salida inválida, prompts y exhaustión; ninguna fuga o ejecución no aprobada.	P0 / G-07
G-14. Pilotos comparativos	Cinco personas en dos sesiones; baseline OpenQodex y solución propuesta; registrar utilidad y abandono.	P0 / G-10
G-15. Evaluar revisión contextual	Prueba ciega con presupuesto y etiquetas, sin modificar resultados deterministas.	P2 / G-14 y corpus

ID y título	Descripción / criterio de cierre	Prioridad / dependencia
G-16. Contribución o decisión de detener	Preparar patch y documentación; contactar upstream solo con autorización; no publicar resultados privados.	P1 / G-14

Ruta crítica: G-02/G-04 → G-05 → G-06/G-07 → G-08–G-10 → G-13/G-14 → decisión. Investigación de usuarios, licencias y diseño de corpus pueden transcurrir en paralelo. Si G-05 o la validación de usuarios falla, evitar seguir construyendo sobre una suposición rota.

15.4 Experimentos de demanda y razones para fallar

Entrevistar 10–15 desarrolladores con experiencia reciente en código generado por IA, evitando describirles la solución al inicio. Preguntar por el último hallazgo, herramientas, tiempo y restricciones de red. En cinco pilotos comparar instalación, hallazgos y tiempo de triaje de OpenQodex y nuestra extensión; permitir abandonar y registrar causa. Pedir una señal conductual: repetir la revisión en otro proyecto, contribuir un caso o solicitar instalación con su equipo. No usar intenciones expresadas como equivalentes de adopción.

Criterios de parar o replantear: la base existente ya resuelve el caso de forma aceptable; pocos usuarios necesitan offline; la configuración excede el tiempo que están dispuestos a dedicar; mejora de ruido exige ocultar verdaderos positivos; mantenimiento de escáneres supera capacidad; la seguridad del runner no es defendible; o el precio humano de revisión excede el valor. La comunidad puede preferir contribuir upstream y no adoptar otro nombre/proyecto. Ninguna de estas hipótesis ha sido medida aquí.

16. Decisión final y próximas acciones

16.1 Respuestas obligatorias

1. **Problema prometedor:** revisar cambios y dependencias en proyectos privados con un flujo determinista, sin red obligatoria, que explique qué cubrió y con qué datos. Es una oportunidad propuesta, no demostrada como la de mayor mercado.

2. **Necesidad:** adopción de IA, resultados de estudios y reportes de ruido apoyan la existencia de los problemas generales [S04–S07, S56–S57]. La necesidad específica de offline debe confirmarse con usuarios.

3. **Herramientas existentes:** OpenQodex, SecGate, OSV-Scanner, Gitleaks, Semgrep, Dependabot y productos comerciales [S08–S19].

4. **Limitaciones observadas:** OpenQodex documenta que omite ciertos escáneres en offline; OSV requiere datos preparados; las coincidencias no prueban alcanzabilidad; herramientas y reglas tienen licencias y soporte distintos.

5. **Diferenciación hipotética:** integrar offline real, fecha y hash de datos y evidencia verificable de cobertura en una experiencia local ya existente. No afirmar exclusividad de snapshot o revisión de push: OpenQodex ya posee funciones en esa dirección.

6. **Bases:** OpenQodex como extensión inicial; OSV-Scanner y Gitleaks; Semgrep CE solo con reglas aceptables; Git y Node. Si no hay extensión viable, evaluar SecGate/adaptador.

7. **Reutilización directa:** ejecutables de escáner, formato OSV y SARIF, selección/reportes y hooks existentes sujetos a comprobación de API y licencia.

8. **Desarrollo:** adaptador offline, control de vigencia y cobertura, pruebas de límites, diseño de snapshot y normalización que realmente falten; contribuir upstream primero.

9. **Arquitectura:** CLI local → snapshot → plan → escáneres aislados → normalización → política → reporte; sin backend obligatorio.

10. **Agentes:** no son necesarios para el MVP; evaluar un revisor opcional solo tras medir utilidad y seguridad.

11. **Tokens:** cero en flujo base; si se activa IA, seleccionar casos, redactar, limitar contexto y presupuesto, cachear con entradas completas y permitir abstención.

12. **Instalación:** componentes fijados, confirmación de red para base inicial y escaneo sin red después; los comandos específicos dependen del spike, por lo que no se presenta un instalador propio como existente.

13. **Prioridad de integración:** CLI/Git y JSON/SARIF; luego CI y compatibilidad con asistentes como consumidores; MCP solo de lectura posteriormente.

14. **Costo:** sin cargos API en A tras preparación; tiempo y hardware sí cuestan. Prototipo 120–160 h y MVP total 320–480 h; ejemplo financiero USD 8,000–24,000 a tarifa supuesta, no cotizada.

15. **Tiempo:** 4 semanas de prototipo evaluable y 8–12 semanas de MVP con dos personas a medio tiempo; sujeto a resultados y puntos de extensión reales.

16. **Riesgos:** demanda no validada, duplicación, dependencia pre-1.0, cobertura insuficiente, datos obsoletos, fuga de datos y explotación del analizador.

17. **Validación:** entrevistas, pilotos frente a base, benchmark etiquetado, pruebas de red/secretos/errores y métricas de cobertura, precisión/recall y tiempo.

18. **Primeras 10 tareas:** G-01 a G-10 en §15.3, en orden de dependencias.

16.2 Tecnologías seleccionadas para el spike

OpenQodex	
Función	CLI, selección, escáneres y reportes
Licencia / repositorio	Apache-2.0
Motivo y condición	Mayor reutilización; confirmar extensión e interfaces pre-1.0.

Node.js + TypeScript	
Función	Runtime y lenguaje del adaptador
Licencia / repositorio	Node.js, TypeScript, revisar licencias concretas
Motivo y condición	Coherencia con la base; Node 22+ según proyecto.

OSV-Scanner	
Función	Dependencias y avisos offline
Licencia / repositorio	Apache-2.0
Motivo y condición	Base local soportada; datos necesitan actualización y verificación.

Gitleaks	
Función	Secretos
Licencia / repositorio	MIT
Motivo y condición	Binario y salida estructurada; redactar evidencia.

Semgrep CE	
Función	Análisis estático opcional del piloto
Licencia / repositorio	LGPL-2.1-or-later
Motivo y condición	Reglas, licencias y cobertura se evalúan por separado.

Git	
Función	Snapshot y control de cambios
Licencia / repositorio	GPL-2.0
Motivo y condición	Herramienta externa; hooks son evitables.

SARIF / JSON	
Función	Intercambio de hallazgos y evidencia
Licencia / repositorio	OASIS SARIF
Motivo y condición	Compatibilidad; especificar versión y extensión de campos.

Sandbox por SO	
Función	Restringir procesos y red
Licencia / repositorio	bubblewrap LGPL, opción Linux

Motivo y condición	Solo si configuración y garantías se prueban en cada plataforma.

Si OpenQodex ya incorpora el modo propuesto antes de empezar, o los usuarios no valoran la brecha, la mejor decisión es utilizarlo o contribuir a él; no hay razón económica para lanzar un duplicado.

17. Bibliografía y fuentes

Las referencias enlazan fuentes originales o fuentes identificadas. Consulta: 28/10/2026. Un repositorio describe sus propias capacidades; los estudios y reportes de comunidad se contextualizan en el texto. Las licencias deben verificarse para cada versión o artefacto distribuido.

- **S01.** OWASP, Top 10 para aplicaciones LLM y GenAI.
- **S02.** Model Context Protocol, recomendaciones de seguridad y seguridad local.
- **S03.** NIST, identidad y autorización para agentes, 2026.
- **S04.** Pearce y otros, Asleep at the Keyboard? Assessing the Security of GitHub Copilot's Code Contributions, IEEE S&P, 2022.
- **S05.** Perry y otros, Do Users Write More Insecure Code with AI Assistants?, ACM CCS, 2023.
- **S06.** Sandoval y otros, Lost at C: A User Study on the Security Implications of Large Language Model Code Assistants, USENIX Security, 2023.
- **S07.** Stack Overflow, Developer Survey 2025: IA.
- **S08.** OpenQodex, repositorio, documentación, limitaciones y benchmark.
- **S09.** SecGate, repositorio y arquitectura.
- **S10.** Nox, repositorio del escáner.
- **S11.** Semgrep, motor comunitario y límites.
- **S12.** Semgrep, repositorio y licencia de reglas.
- **S13.** Aqua Security, Trivy.
- **S14.** Google, OSV-Scanner.
- **S15.** Gitleaks, repositorio.
- **S16.** GitHub, CodeQL CLI.
- **S17.** SonarSource, SonarQube Community Build.
- **S18.** Snyk, planes y precios.
- **S19.** GitHub, Dependabot alerts.
- **S20.** Renovate, CLI, licencia y funcionamiento.
- **S21.** Bridgecrew, Checkov.
- **S22.** OpenSSF, Scorecard.
- **S23.** ZAP, repositorio.
- **S24.** OpenClaw, GHSA-g55j-c2v4-pjcg.
- **S25.** Anthropic, GHSA-4fgq-fpq9-mr3g.
- **S26.** Cline, post mortem de publicación npm y GitHub, GHSA-9ppg-jx86-fqw7.
- **S27.** Cursor, GHSA-v64q-396f-7m79.
- **S28.** ToolHive, GHSA-h4mf-84xq-q2fc.
- **S29.** OWASP, seguridad de agentes y MCP.
- **S30.** OpenAI, arquitectura de Codex y seguridad de entornos.
- **S31.** Anthropic, contención y límites de confianza.
- **S32.** Cursor, seguridad del agente.
- **S33.** Gemini CLI, sandbox y opciones.
- **S34.** GitHub, sandboxes local y cloud de Copilot.
- **S35.** OpenCode, repositorio.

- **S36.** Aider, repositorio.
- **S37.** Cline, repositorio y SDK.
- **S38.** Roo Code, repositorio archivado.
- **S39.** Qwen, ficha de Qwen3-4B.
- **S40.** OWASP, Benchmark para AppSec.
- **S41.** SastBench, benchmark de triaje SAST, preprint 2026.
- **S42.** SWE-bench, benchmark de ingeniería de software.
- **S43.** OWASP, AppSec Agent.
- **S44.** Pipelock, repositorio y licencias core/enterprise.
- **S45.** Stacklok, ToolHive.
- **S46.** OpenAI, precios API.
- **S47.** Google, precios Gemini API.
- **S48.** GitHub, facturación de Actions.
- **S49.** Telegram, FAQ de bots y límites.
- **S50.** OSV, GHSA-35jh-r3h4-6jhm / CVE-2021-23337.
- **S51.** Google, OSV-Scanner: modo offline.
- **S52.** Open Source Initiative, licencias aprobadas.
- **S53.** SLSA, especificación v1.2.
- **S54.** OpenVEX, formato y proyecto.
- **S55.** DORA, reporte 2025 sobre IA y desarrollo.
- **S56.** Semgrep, issue de falso positivo en taint; OSV-Scanner, issue sobre resultado para paquete local.
- **S57.** Reddit, discusión sobre backlog y alertas CVE, experiencia individual, no evidencia representativa.
- **S58.** Anthropic, How we contain Claude across products.
- **S59.** OSI, Open Source AI Definition 1.0.
- **S60.** LangChain, LangGraph.
- **S61.** Microsoft, AutoGen archivado.
- **S62.** Model Context Protocol, SDK oficial de Python.
- **S63.** Open Policy Agent, repositorio.
- **S64.** Containers, bubblewrap y política de seguridad.
- **S65.** OpenQodex, modelo de seguridad y confianza.
- **S66.** OWASP, Application Security Verification Standard.
- **S67.** GitHub, autofix de code scanning.
- **S68.** Ollama, Qwen3 para ejecución local.
- **S69.** Discord, límites de API para desarrolladores.
- **S70.** GitHub, términos de CodeQL CLI.
- **S71.** OWASP, MCP Security Cheat Sheet.
- **S72.** MITRE, ATLAS.
- **S73.** OpenAI, Codex Security; GitHub, Copilot Autofix.
- **S74.** OpenClaw, repositorio y licencia.

• **S75.** Hacker News, discusión sobre revisión de código y falsos positivos, opiniones y resultados no reproducidos.

18. Apéndice: reproducir la prueba local ejecutada

Requisitos: Python 3 y Git. Copiar el siguiente programa en `experiment.py` y ejecutar `python3 experiment.py`. Crea un directorio temporal, no accede a red y no ejecuta el contenido de los archivos analizados. La evidencia de OSV está incorporada como coincidencia exacta transcrita, no como base de producción. Los tiempos variarán.

```
import ast
import json
import re
import subprocess
import tempfile
import time
from concurrent.futures import ThreadPoolExecutor
from pathlib import Path

ADVISORY = {
    'id': 'GHSA-35jh-r3h4-6jhm',
    'alias': 'CVE-2021-23337',
    'package': 'lodash',
    'affected_version': '4.17.20',
    'fixed_version': '4.17.21',
    'source': 'https://osv.dev/vulnerability/GHSA-35jh-r3h4-6jhm',
}

def call_git(repo, *args):
    proc = subprocess.run(
        ['git', '-C', str(repo), *args],
        check=True, text=True,
        stdout=subprocess.PIPE, stderr=subprocess.PIPE,
    )
    return proc.stdout

def scan_eval(repo, files):
    out = []
    for name in files:
        path = repo / name
        if not name.endswith('.py') or not path.is_file():
            continue
        for n in ast.walk(ast.parse(path.read_text())):
            if (isinstance(n, ast.Call)
                and isinstance(n.func, ast.Name)
                and n.func.id == 'eval'):
                out.append({
                    'scanner': 'toy-ast-eval', 'path': name,
                    'line': n.lineno, 'type': 'candidate-eval',
                })
    return out

def scan_secret(repo, files):
    out = []
    for name in files:
        path = repo / name
        if not name.endswith('.py') or not path.is_file():
            continue
        for i, line in enumerate(path.read_text().splitlines(), 1):
            if re.search(r'DEMO_API_KEY\s*=\s*["\x27]DEMO_[A-Z0-9]+["\x27]', line):
                out.append({
                    'scanner': 'toy-secret', 'path': name,
                    'line': i, 'type': 'synthetic-key-pattern',
                })
    return out

def scan_lock(repo, files):
    if 'package-lock.json' not in files:
```

```
        return []
    lock = json.loads((repo / 'package-lock.json').read_text())
    results = []
    for path, package in lock['packages'].items():
        if (path == 'node_modules/lodash'
            and package.get('version') == ADVISORY['affected_version']):
            results.append({
                'scanner': 'toy-offline-advisory',
                'path': 'package-lock.json', 'type': 'known-version-match',
                'advisory': ADVISORY['id'], 'version': package['version'],
            })
    return results

def run(repo, files):
    start = time.perf_counter()
    with ThreadPoolExecutor(max_workers=3) as pool:
        jobs = [pool.submit(fn, repo, files) for fn in (scan_eval, scan_secret, scan_lock)]
        hits = [hit for job in jobs for hit in job.result()]
    return {
        'files': files,
        'findings': sorted(hits, key=lambda x: (x['scanner'], x['path'])),
        'elapsed_ms': round((time.perf_counter() - start) * 1000, 2),
    }

with tempfile.TemporaryDirectory(prefix='security-research-poc-') as tmp:
    repo = Path(tmp)
    call_git(repo, 'init', '-q')
    call_git(repo, 'config', 'user.name', 'Experiment')
    call_git(repo, 'config', 'user.email', 'experiment@example.invalid')
    (repo / 'old.py').write_text('def old(value):\n    return eval(value)\n')
    (repo / 'app.py').write_text('def calculate(value):\n    return value\n')
    lock = {
        'name': 'synthetic', 'lockfileVersion': 3,
        'packages': {'': {}, 'node_modules/lodash': {'version': '4.17.21'}},
    }
    (repo / 'package-lock.json').write_text(json.dumps(lock, indent=2) + '\n')
    call_git(repo, 'add', '.')
    call_git(repo, 'commit', '-qm', 'Synthetic baseline')
    (repo / 'app.py').write_text(
        'DEMO_API_KEY = "DEMO_ABCD1234"\n'
        'def calculate(value):\n    return eval(value)\n'
    )
    lock['packages']['node_modules/lodash']['version'] = '4.17.20'
    (repo / 'package-lock.json').write_text(json.dumps(lock, indent=2) + '\n')
    changed_raw = call_git(repo, 'diff', '--name-only', '-z', 'HEAD')
    changed = sorted(x for x in changed_raw.split('\0') if x)
    all_files = sorted(x for x in call_git(repo, 'ls-files', '-z').split('\0') if x)
    result = {
        'advisory_source': ADVISORY['source'],
        'full': run(repo, all_files),
        'incremental': run(repo, changed),
        'limitations': (
            'Synthetic detectors; no production scanner or live API; '
            'no reachability inference.'
        ),
    }
    print(json.dumps(result, indent=2, ensure_ascii=False))
```